Smart Study Champion: Adaptive Exam Practice And Interview Preparation Platform

A Report Submitted
in Partial Fulfillment for the Degree of

Bachelor of Engineering
in
Information Technology
(Session 2026–2027)

Submitted By:
Shravika Jain (23I7166)
Taniksha Patidar (23I7173)
Siddhi Nagar (23I7186)

Guided By:
Dr. Hemant Makwana

Department of Information Technology
Institute of Engineering and Technology
DAVV, Indore (M.P.)

October 2026

CERTIFICATE

The project report entitled **"Smart Study Champion: Adaptive Exam Practice And Interview Preparation Platform"** submitted by **Shravika Jain, Taniksha Patidar, and Siddhi Nagar** is approved for partial fulfillment for the award of Bachelor of Engineering in **Information Technology** degree by Devi Ahilya Vishwavidyalaya, Indore.

Internal Examiner	External Examiner

RECOMMENDATION

The dissertation entitled **"Smart Study Champion: Adaptive Exam Practice And Interview Preparation Platform"** submitted by **Shravika Jain, Taniksha Patidar, and Siddhi Nagar** is a satisfactory account of the bona fide work done under my supervision and is recommended towards the partial fulfillment for the award of Bachelor of Engineering in **Information Technology** degree by Devi Ahilya Vishwavidyalaya, Indore.

Date: 27/10/2026

_____

Dr. Hemant Makwana
Department of Information Technology

_____

Counter-signature of HOD

CANDIDATE DECLARATION

We hereby declare that the work presented in this project entitled **"Smart Study Champion: Adaptive Exam Practice And Interview Preparation Platform"**, in partial fulfillment of the degree of Bachelor of Engineering in **Information Technology**, is an authentic record of work carried out under the supervision and guidance of **Dr. Hemant Makwana**, Department of **Information Technology**, Institute of Engineering and Technology, Devi Ahilya Vishwavidyalaya, Indore.

We are responsible for the material embodied in this project. This project has not been submitted, in whole or in part, for the award of any other degree or diploma. All external sources used in preparing this report are acknowledged in the references.

Date: 27/10/2026
Place: Indore

_____
Shravika Jain

_____
Taniksha Patidar

_____
Siddhi Nagar

SELF-DECLARATION OF PLAGIARISM REPORT

The project report entitled **“Smart Study Champion: Adaptive Exam Practice And Interview Preparation Platform”** submitted by **Shravika Jain, Taniksha Patidar, and Siddhi Nagar** in partial fulfillment of the requirements for the award of Bachelor of Engineering in **Information Technology** has been checked for plagiarism using a standard detection tool.

The similarity index of the report is _____ %, which is within the permissible limit as per institute/university norms.

We declare that the work submitted is original, that materials from other sources have been cited, and that we accept responsibility for any discrepancy identified in the future.

Signature of Student(s):

1. _____ (Shravika Jain)
2. _____ (Taniksha Patidar)
3. _____ (Siddhi Nagar)

Date: _____

ACKNOWLEDGEMENTS

We express our sincere gratitude to our project guide and HOD of the department, **Dr. Hemant Makwana**, for the guidance, technical direction and constructive feedback provided throughout the development of Smart Study. The discussions on project scope, software design and evaluation helped us refine the system and this report.

We are also grateful to the Director and the Institute of Engineering and Technology, Devi Ahilya Vishwavidyalaya, for providing the institutional environment in which this work was undertaken.

We acknowledge the maintainers of the open-source frameworks and libraries used in this project, including Spring Boot, FastAPI, PostgreSQL, scikit-learn and Faster-Whisper. Their documentation and software made it possible to implement and test the application. Finally, we thank our families and peers for their encouragement during the project.

ABSTRACT

Students preparing for examinations often have to locate recurring topics across past papers, create additional practice questions and judge what to study next. Smart Study is a web-based learning application that brings these activities into one workflow. A student organizes work by subject and topic, uploads past examination papers in PDF format, and uses extracted text to identify question themes and recurring questions. The application can generate original multiple-choice or subjective practice questions from a selected topic and available study material. Its quiz service records responses and adjusts an ability estimate so that the next question is selected near the learner's current estimated level. Learners can practise multiple-choice questions, subjective questions or both from a stored question bank. Subjective responses receive model-assisted scores and feedback together with an advisory estimate of whether the text was AI-written, and every test is saved so that answers, grades and feedback can be reviewed later. A separate authenticity check estimates AI-written content in learner-supplied notes or essays. An interview-practice workflow supports academic-subject or engineering-role sessions, typed responses and browser-recorded speech transcribed by a local Whisper model in the ML container. The system is implemented as a browser frontend, a Spring Boot API, a Python FastAPI machine-learning service and a PostgreSQL database, deployed together with Docker Compose. This report presents the problem, related learning systems, requirements, design, data model, implementation and test evidence. Fresh automated test runs (67 ML-service tests and 53 backend tests, all passing) and end-to-end runs against the deployed services are reported, together with a small evaluation showing that open AI-writing detectors identify text from a current language model only partially. Smart Study is a practice and revision aid, not a replacement for instructor judgment or an accredited examination platform. Its usefulness depends on the quality of uploaded material and generated feedback, and generated content should be reviewed by learners and instructors.

Keywords: adaptive learning, examination preparation, question generation, item response theory, past-paper analysis, interview practice, AI-generated text detection, self-evaluation.

TABLE OF CONTENTS

Certificate i
Recommendation ii
Candidate Declaration iii
Self-Declaration of Plagiarism Report iv
Acknowledgements v
Abstract vi
Chapter 1 — Introduction 1
1.1 Overview and Issues Involved 1
1.2 Problem Definition 1
1.3 Proposed Solution 2
Chapter 2 — Literature Survey 4
2.1 Existing Solutions 4
2.2 Methodology 6
Chapter 3 — Analysis and Design 9
3.1 Software Requirements 9
3.2 Hardware Requirements 11
3.3 Analysis Diagrams 12
3.4 Design Diagrams 15
Chapter 4 — Implementation and Testing 17
4.1 Database Design 17
4.2 Class Diagram 22
4.3 Test Cases 24
Chapter 5 — Conclusion 55
References 57
Appendix 59

CHAPTER 1
INTRODUCTION

1.1 Overview and Issues Involved

Past examination papers are valuable because they expose learners to the wording, topic emphasis and level of questions used in earlier assessments. In practice, however, a student may have to collect papers from several locations, read them manually, notice repeated concepts, prepare a personal question bank and then decide which question to attempt next. These steps are time-consuming and do not automatically provide targeted feedback. A static list of questions also gives every learner the same sequence even when their knowledge differs.

Smart Study addresses this workflow through a browser application organized around subjects and topics. Users can maintain their own subject areas, upload past papers, inspect recurring themes, generate new practice questions, take a quiz that selects questions according to an evolving ability estimate, and rehearse academic or engineering interviews. The system combines deterministic software components (storage, parsing, topic organization, quiz state and ability updates) with machine-learning services for text analysis, question generation, written feedback and speech transcription.

The application is designed as a study aid. Text extraction may be imperfect for scanned or poorly formatted documents; semantic clustering can group questions that share vocabulary without being conceptually identical; and generated questions or scores may contain errors. Consequently, the application should present its outputs as guidance and preserve the learner's responsibility to verify educational content.

1.2 Problem Definition

University learners lack a single, account-based workflow that converts their own past-paper material into organized, recurring-topic insights and adaptive practice while also supporting written-answer feedback and interview rehearsal. Manual analysis is repetitive; static practice does not adapt to demonstrated performance; and separate tools often divide content management, practice and feedback across different interfaces.

Project objective. Design and implement a deployable web system that securely organizes study content, extracts and analyzes uploaded examination papers, generates topic-linked practice, adapts quiz difficulty from responses, and supports conversational interview practice with optional voice input.

Project objectives

- Provide account registration, login and user-scoped access to subjects and study material.

- Allow learners to create subjects/topics and upload PDF question papers with an optional examination year.
- Extract text from digital PDFs and route scanned documents to a protected OCR service.
- Group extracted questions and report recurring questions and topic clusters across papers.
- Generate original multiple-choice and subjective questions and conduct adaptive quizzes.
- Let learners choose multiple-choice, subjective or both question types when starting a test, and save every test for later self-evaluation.
- Provide rubric-guided feedback for subjective answers and conduct subject/role interview sessions.
- Estimate, as an advisory signal, whether subjective answers and learner-supplied material were written by AI.
- Support browser audio capture and local-service transcription without sending interview audio to an external speech provider.

Scope and constraints

The implemented scope is a single-page browser interface, Spring Boot REST API, FastAPI ML service and PostgreSQL-backed storage, with Docker Compose as the local deployment route. The ML text-generation integration uses an Ollama-compatible endpoint configured through environment variables. Speech recognition uses Faster-Whisper in the ML container. A working provider key and configured services are required for model-backed features. No claim is made that the model outputs are guaranteed correct, that the system is a high-stakes assessment tool, or that the current repository represents a production-scale hosted service.

1.3 Proposed Solution

Smart Study uses a service-oriented design. The frontend presents sign-in, subject dashboards, paper management, practice, test-history, authenticity-check and interview views. The backend authenticates requests, checks ownership, persists domain data and coordinates calls to the ML service. The ML service exposes typed HTTP endpoints for analysis, question generation, grading, interview turns, AI-writing detection, scanned-PDF OCR and audio transcription. PostgreSQL stores structured entities; uploaded PDFs are kept in a persistent uploads volume. The system is composed with Docker so that its service boundaries and dependencies can be started together.

The paper-analysis pipeline extracts question-like text, normalizes vocabulary, creates TF-IDF features, combines them with MiniLM sentence embeddings so that reworded questions on the same topic can be grouped, and clusters questions using K-means. Candidate cluster counts are compared using the silhouette score within a configured bound. Similar questions across different papers are joined using TF-IDF cosine similarity and a configurable threshold; repeated concepts

are returned with occurrence and year metadata. This produces an inspection aid rather than a guarantee of syllabus coverage.

Adaptive practice maintains an ability estimate for a quiz session. The implementation uses a logistic response probability and a bounded update after each answer, then selects a remaining question whose difficulty is closest to the current estimate. This is a simplified item-response-inspired procedure; it is not a calibrated psychometric instrument and does not estimate all parameters of a full item response model.

For self-evaluation, every test is stored with the learner's answers. A learner can start a test limited to multiple-choice questions, subjective questions or both, and later reopen any past test to compare each answer with the correct option or the model answer and marking guide, together with the grade and feedback. Written answers also receive an AI-writing estimate from a locally run classifier. The same detector powers a standalone authenticity check for notes or essays, which reports an overall estimate and section-by-section scores. Because such detectors are imperfect (Section 4.3), the estimate is advisory and never changes a grade.

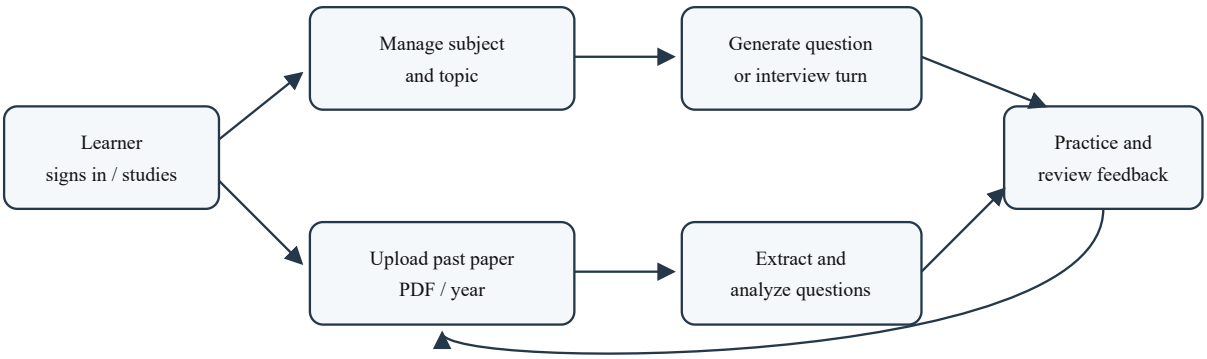


Figure 1.1: High-level study workflow. Generated and analyzed material returns to a learner-controlled practice and review loop.

Expected contribution

The project's contribution is the integration of paper organization, recurring-question analysis, question generation, adaptive practice and interview rehearsal in one subject-centered application. Compared with static question lists, the quiz loop responds to prior answers; compared with standalone model prompts, the workflow stores user-owned topics, papers, questions and quiz sessions in a structured application. These are design-level advantages; comparative learning outcomes have not been established by a controlled educational study.

CHAPTER 2
LITERATURE SURVEY

2.1 Existing Solutions

Digital learning products commonly provide some combination of structured lessons, flashcards, assessment authoring, question banks and progress tracking. The following review compares representative tools and categories relevant to Smart Study. It is a functional comparison based on publicly described product capabilities, not a benchmark or independent evaluation of learning effectiveness.

Learning management systems and Moodle Quiz

Moodle is an open-source learning management system used to deliver course content, activities and assessments. Its Quiz activity supports instructor-authored questions, attempts, grading and feedback within a course [6]. It is a capable platform when an institution configures the course and question bank. Smart Study differs in its focus on an individual learner's past examination papers, automatic cross-paper topic/repetition analysis and adaptive practice workflow. Moodle's extensive administration model may be more than a learner needs for personal revision.

Quizlet

Quizlet centers on user- or teacher-created study sets and offers digital flashcard and practice experiences [7]. Its strength is quick vocabulary and concept review supported by a large content ecosystem. A user typically needs prepared study material; it does not constitute the project's specific pipeline for parsing a learner's examination-paper collection, clustering paper questions and using those patterns to seed topic-linked adaptive quiz sessions.

Khan Academy

Khan Academy provides structured lessons and practice across supported subjects, with learner progress linked to a curated curriculum [8]. Its strength is the combination of instructional material and exercises. Smart Study is instead designed to start from a learner's own examination papers and topic organization. It does not attempt to reproduce Khan Academy's broad curriculum, instructional video library or institution-scale learning analytics.

Commercial exam-preparation platforms

Exam-preparation services commonly provide syllabus-aligned courses, mock examinations, question banks and performance reports for particular examinations. Their curated content can provide consistency and exam-specific guidance. Their scope is often tied to a provider's content catalog and target assessment. Smart Study's intended distinction is learner-supplied paper analysis and customizable subjects rather than a centrally curated, examination-specific question bank.

AI study assistants and question generators

General-purpose AI assistants and educational question generators can create explanations, practice prompts and conversational feedback. They offer flexible interaction but depend on the supplied context and the model's reliability. A standalone chat does not automatically provide a persistent relation between a learner's subjects, uploaded papers, generated question records, quiz state and past-paper recurrence. Smart Study wraps model services in a structured subject-and-quiz workflow and validates generated question fields before storing them.

Speech-enabled interview practice

Interview simulators and voice assistants can help candidates rehearse responses. Smart Study's interview workflow supports subject or engineering-role targets, question sequences and feedback; the browser can capture audio and submit it to the local ML service for transcription. This is a focused practice workflow, not a hiring assessment, professional interview credential or guarantee of interview performance.

AI-generated text detectors

Detectors of AI-generated text are commonly trained as classifiers on pairs of human-written and model-written text. A widely used early example was trained on the HC3 corpus of human and ChatGPT answers [16]. Such supervised detectors tend to generalize poorly to text from newer models, and they have been shown to misclassify writing by non-native English speakers as AI-generated [17]. More recent open detectors, such as Fakespot's RoBERTa model [18], are trained on output from a wider range of models. Smart Study uses a detector of this kind to give learners a reflective signal about their own writing, presents it as a probability with explicit uncertainty, and does not use it for grading.

Research concepts informing the project

Item response theory models the probability of a response in relation to learner ability and item properties. The project borrows the intuition that practice can be selected near an evolving estimate of ability, while implementing a deliberately simpler bounded logistic update [1, 2]. TF-IDF assigns greater importance to terms that are informative within a document collection, while K-means groups vector representations around centroids [3, 4]. The silhouette coefficient can help compare candidate clusterings by considering compactness and separation [5]. Sentence embeddings represent a whole sentence as a dense vector so that sentences with similar meaning lie close together even when they share few words [19]; the compact MiniLM family makes this practical on a CPU [20]. These techniques are used as practical analysis components; none alone understands educational semantics or proves that a cluster is pedagogically coherent.

Gap and justification. The reviewed systems typically emphasize curated courses, flashcard sets, instructor-managed assessments or open-ended generative chat. The project addresses a narrower integration gap: a learner can bring their own paper corpus, organize it by subject, inspect

recurring patterns and continue directly into generated and adaptive practice, with interview rehearsal in the same application. The gap is functional and architectural; user studies are still required to demonstrate learning benefit.

2.2 Methodology

Development methodology

The project follows an iterative implementation approach. First, the repository's user-facing workflow and domain objects are defined. Next, API contracts and persistence entities are implemented in the Spring Boot service. The Python service supplies analysis and model-backed tasks behind typed request/response schemas. The browser interface consumes the API, while Docker Compose connects the application and database. Automated unit, integration and API tests are used to check important behaviors. Iterations address the paper-import path, adaptive quiz lifecycle, interview turn flow and voice transcription. Later iterations added question-type selection, stored test history and AI-writing detection, and hardened the language-model integration after live testing against the hosted model. Configuration values and secrets are supplied through environment variables rather than committed source.

Concepts and algorithms

- **REST service:** An HTTP API exposes resources and actions using JSON or multipart form data. The frontend communicates with the backend; the backend calls the internal ML service.
- **PDF extraction and OCR:** PDFBox extracts embedded text from digitally generated documents. When scanned pages contain no usable text, an authenticated internal endpoint can render pages and run OCR. Extraction quality depends on document clarity and language support.
- **TF-IDF:** Each question is represented by weighted one- and two-word terms. Frequent general examination words are removed to reduce uninformative similarity.
- **Sentence embeddings:** Each question is also encoded with the all-MiniLM-L6-v2 model (mean-pooled and L2-normalized, pinned revision). For clustering, the TF-IDF and embedding vectors are joined with weights of 0.4 and 0.6, so cosine similarity reflects both shared wording and meaning. In a calibration on sample algorithm questions, the embeddings scored the reworded pair "N-Queens" / "8 queens puzzle" at 0.66 where TF-IDF scored 0.16, but also scored two different shortest-path questions at 0.62; recurring-question detection therefore remains on TF-IDF. If the embedding model is unavailable or still loading, clustering falls back to TF-IDF alone.
- **K-means and silhouette selection:** K-means groups the combined text vectors. The analysis chooses a cluster count by checking silhouette scores for a bounded set of candidates; deterministic random seeds make the process more reproducible.

- **Cosine similarity and repeated questions:** Questions whose TF-IDF cosine similarity exceeds a threshold are linked only across different paper IDs. Connected groups supply the repeated-question list and years.
- **Adaptive quiz:** A logistic probability estimates the chance of a correct response given ability and difficulty. After an answer, the estimate is updated by a diminishing step and clamped to a supported range. The next eligible item is the one closest to the current estimate.
- **Structured generation and feedback:** The ML service requests schema-constrained JSON. In live testing, the hosted model (gpt-oss:120b on Ollama Cloud) did not enforce the requested schema: it returned bare lists, renamed fields or Markdown. The service therefore also states the schema in the prompt, parses replies leniently, maps common alternative field names, retries once on malformed output, shuffles multiple-choice options so the correct answer's position varies, and discards duplicate prompts or options before validating question type, option count, correct-answer index and reference answer. Subjective feedback is model-assisted and should be treated as formative feedback.
- **AI-writing detection:** Text is split into overlapping 512-token windows and classified by fakespot-ai/roberta-base-ai-text-detection-v1 (pinned revision). Window probabilities are averaged and the text is flagged at 0.60. The interface reports three bands: below 35% likely human-written, 35–60% uncertain, and 60% or above likely AI-written. Long documents also show the score of each window.
- **Background model loading:** The detector, Whisper and embedding models load on background threads at start-up. A request that arrives first never waits for a download: clustering falls back to TF-IDF, and detection or transcription report that the model is still loading.
- **Speech transcription:** The browser records an audio response and sends it to the internal API. Faster-Whisper performs local CPU transcription in the ML container; the resulting text is submitted as the candidate response.

Security and privacy considerations

Authenticated application routes use JWT-based identity and user ownership checks. All ML-service endpoints except the health check are protected by a shared internal service key and constant-time comparison. Deployment secrets, database credentials and model-provider keys are configured externally. Uploaded papers and audio contain potentially sensitive information, so operators should restrict access to volumes, use HTTPS outside localhost, establish retention rules, and avoid uploading material without the right to use it. Model providers may process text sent for generation or grading; operators must review provider terms and avoid sensitive personal data. The local transcription path is designed not to send captured interview audio to an external speech provider. The authenticity check stores only the source name, character count and scores; the submitted text is not persisted. Because AI-writing detectors can wrongly flag human writing,

including that of non-native English writers [17], the application labels its estimate as advisory and never uses it for grading.

CHAPTER 3
ANALYSIS AND DESIGN

3.1 Software Requirements

Functional requirements

ID	Requirement	Acceptance description
FR-01	Account access	A visitor can register or sign in; protected actions are associated with the authenticated user.
FR-02	Subject and topic management	A learner can create subjects and topics and view their own study areas.
FR-03	Paper management	A learner can upload a PDF to a subject, optionally record its exam year, list papers and delete a paper.
FR-04	Paper analysis	Text from uploaded papers is analyzed into question clusters and cross-paper recurring-question results.
FR-05	Question generation	The system can request multiple-choice, subjective or both kinds of questions for selected topics and persist usable results.
FR-06	Adaptive quiz	The learner can choose multiple-choice, subjective or both question types from the subject's question bank, answer adaptive quiz questions, receive results/feedback and progress until completion.
FR-07	Subjective assessment	A written response is evaluated against a reference answer and rubric, returning a bounded score and feedback.
FR-08	Interview practice	A learner can choose an academic subject or engineering role, answer a question sequence and receive feedback.
FR-09	Voice response	A learner can record a response in the browser and submit audio for transcription by the local ML service.
FR-10	Test history and review	Every test is stored. The learner can list past tests and review each question with their answer, the correct option or model answer and marking guide, grade, feedback and AI-writing estimate.
FR-11	AI-writing estimate	Each subjective answer receives an advisory probability that it was AI-written, stored with the answer and shown without affecting the grade. If the detector is unavailable, the answer is still graded and the interface says so.
FR-12	Authenticity check	A learner can paste text or upload a PDF, .txt or .md file (50–60,000 characters) and receive an overall AI-writing estimate with section-by-section scores. Results are kept in the learner's history; the text is not stored.

Non-functional requirements

- **Security:** User-owned data must be checked against the authenticated principal; internal ML endpoints must not be exposed as public application routes.

- **Reliability:** Invalid PDF, malformed model responses and unavailable dependencies must produce explicit errors rather than fabricated success results. Model downloads happen in the background so that requests are not blocked, and grading continues without an AI estimate if the detector is unavailable.
- **Usability:** Primary workflows must be available in a modern browser, with typed answers retained as an alternative to voice. Nginx asks browsers to revalidate the application files on each load so that updates take effect immediately.
- **Transparency:** AI-writing estimates are shown as probabilities with an uncertainty band and a statement that they are not proof of authorship.
- **Maintainability:** Frontend, API, ML service and persistence responsibilities are separated and communicate through explicit contracts.
- **Portability:** Dockerfiles and Docker Compose describe the local multi-service deployment; secrets and runtime configuration remain external.
- **Performance:** File-size and question-count limits bound expensive operations. Exact throughput and latency targets have not yet been established by load testing. Slow ML calls (OCR, detection, grading and generation) run outside database transactions, with open-session-in-view disabled, so a request does not hold a database connection while it waits.
- **Data integrity:** Relational ownership and foreign-key associations keep each subject, paper, topic and quiz connected to its user context.

Technologies and tools

Component	Technology	Reason for selection
User interface	HTML, CSS, JavaScript, Nginx	Lightweight browser client; static assets are served separately from API processing.
Application API	Java 25, Spring Boot, Spring Web MVC, Spring Data JPA, Spring Security	Typed REST endpoints, persistence integration and authentication support.
Persistence	PostgreSQL 16	Relational storage for users, subjects, papers, questions and quiz sessions.
ML service	Python, FastAPI, Pydantic, scikit-learn, Hugging Face Transformers, PyTorch (CPU)	Typed service contracts, established text-vectorization and clustering implementations, and local transformer models.
PDF	Apache PDFBox; pdf2image and OCR dependencies for scans	Extract embedded PDF text and provide a separate OCR path for scanned documents.
Language model	Ollama-compatible API	Question generation, subjective feedback and interview interaction are configured independently of application code.

Component	Technology	Reason for selection
Speech	Faster-Whisper	Local-container transcription so audio does not require an external speech provider.
Semantic features	sentence-transformers/all-MiniLM-L6-v2 (pinned revision)	Small sentence-embedding model that groups reworded questions on the same topic [19, 20].
AI-writing detection	fakespot-ai/roberta-base-ai-text-detection-v1 (pinned revision)	Locally run classifier, chosen after comparing three open detectors (Section 4.3) [18].
Deployment	Docker, Docker Compose	Build and connect the frontend, backend, ML service and database consistently.
Testing	JUnit / Spring Boot Test; pytest	Unit, integration and API-level checks for Java and Python components.

3.2 Hardware Requirements

Development/deployment host (minimum practical baseline): 64-bit Windows, macOS or Linux capable of running Docker Desktop/Engine; a multi-core CPU; 8 GB RAM for a small local trial (16 GB or more recommended when PostgreSQL, the Java service and the local models run together); several GB of free disk for images, uploaded papers and the downloaded local models (about 500 MB for the AI-writing detector, 90 MB for the sentence-embedding model and 145 MB for the Whisper base model); and a microphone for voice practice. On slow connections the first model download can take a long time; the models are cached in a Docker volume and can be pre-seeded from an existing Hugging Face cache. The exact memory and disk requirement depends on the chosen Whisper model, OCR packages and concurrent workload.

Client device: A recent laptop, desktop or mobile device with a standards-compliant browser and network access to the application. Microphone access requires a secure context such as localhost or HTTPS and explicit browser permission. If the client microphone is unavailable, interview answers can be typed.

External services: The configured Ollama endpoint and model must be reachable for question generation, grading and conversational interview responses. When using a hosted provider, a valid API key is required. These are operational dependencies, not hardware requirements of the learner’s browser.

3.3 Analysis Diagrams

3.3.1 Use Case Model

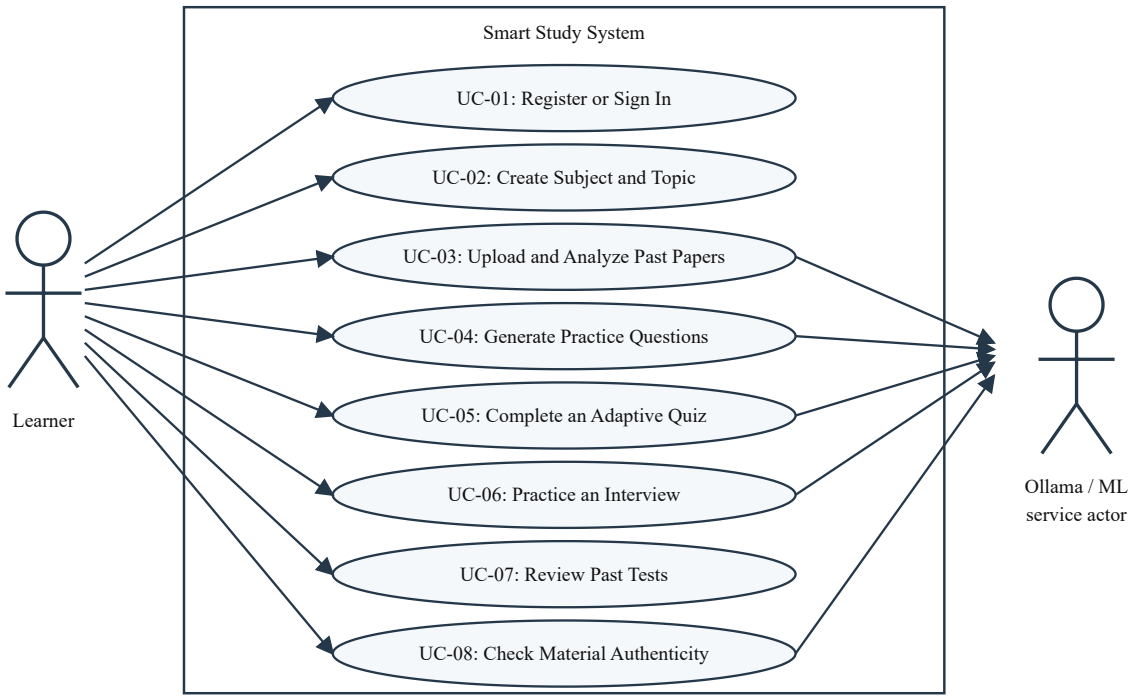


Figure 3.1: Use Case Model of Smart Study (UC-01 to UC-08).

The learner initiates all primary use cases. The backend mediates access to persistent data and calls the ML service only for operations that need analysis, generation, grading, AI-writing detection, interview responses or transcription. Reviewing past tests uses stored data only. The provider is therefore a supporting service, not an independent user.

3.3.2 Use Case Description

UC-01 — Register or Sign In

Primary actor: Learner. **Preconditions:** Application is reachable; learner has a valid email and password for the selected action. **Trigger:** Learner submits the registration or login form. **Main flow:** (1) System validates required fields. (2) For registration, the service creates the account if the email is not already in use; for login, it verifies credentials. (3) On success, the service issues an authentication token and the client opens the subject dashboard. **Alternatives:** Invalid fields, duplicate email or rejected credentials return an error and leave the user unauthenticated. **Postconditions:** Successful authentication establishes the user context for subsequent requests.

UC-02 — Create Subject and Topic

Primary actor: Authenticated learner. **Preconditions:** Valid authenticated session. **Trigger:** Learner submits a subject or topic form. **Main flow:** (1) Client sends the name and related parent identifier. (2) Backend validates the request and confirms parent ownership. (3) Backend stores the new record and returns it to the client. **Alternatives:** Missing/invalid names or a resource owned by another user are rejected. **Postconditions:** The subject/topic is available within that learner's workspace.

UC-03 — Upload and Analyze Past Papers

Primary actor: Authenticated learner. **Preconditions:** Learner owns a subject; input is a supported PDF within configured upload limits. **Trigger:** Learner uploads a paper and optionally supplies its year. **Main flow:** (1) Backend verifies subject ownership and stores the file under a generated name. (2) Text is extracted from the PDF; if the normal extraction path yields no usable text, the configured OCR flow may be used. (3) Paper metadata and text are associated with the subject. (4) On analysis request, extracted questions are normalized, vectorized, clustered and compared across papers. (5) Cluster and recurrence results are returned. **Alternatives:** Invalid PDF, unreadable scan, no papers or unavailable ML service produce an explicit error. Failed upload processing removes the newly stored file where possible. **Postconditions:** A successful upload is attached to its subject; analysis is returned without changing the source paper.

UC-04 — Generate Practice Questions

Primary actor: Authenticated learner. **Preconditions:** Learner owns a subject and has selected one or more topics. **Trigger:** Learner requests a practice set. **Main flow:** (1) Backend composes a typed generation request with subject, topics, requested question type and optional paper context. (2) ML service requests structured output from the configured language model. (3) Response fields and question-specific rules are validated. (4) Usable questions are stored against their topic and returned. **Alternatives:** Provider unavailable, malformed output or no valid questions are reported as failure; invalid model items are not silently presented as questions. **Postconditions:** Valid questions are available for later practice.

UC-05 — Complete an Adaptive Quiz

Primary actor: Authenticated learner. **Preconditions:** Learner owns the subject; a usable question pool exists. **Trigger:** Learner chooses multiple-choice, subjective or both and a question count, then starts a test. **Main flow:** (1) Backend filters the subject's question bank by the chosen type, creates a quiz session and selects the initial question. (2) Learner submits an answer. (3) Backend records correctness, updates ability and answer counts, and chooses the next question closest to current ability. (4) System returns answer feedback and progress. (5) Session ends at its configured maximum or when no questions remain. **Alternatives:** An invalid session/question/answer is rejected; if no question of the chosen type exists, the learner is told to generate some first. Subjective answers are graded and checked for AI-writing outside the database transaction (Figure 3.4); if the detector is unavailable the answer is graded without an estimate. **Postconditions:** Completed session records its status, counts, final estimate and timestamps, and every answer is retained for later review.

UC-06 — Practice an Interview

Primary actor: Authenticated learner. **Preconditions:** Interview service and, for voice, browser microphone permission are available. **Trigger:** Learner chooses a subject or engineering role, level and question count. **Main flow:** (1) Client requests the next interview turn with prior conversation context. (2) ML service returns a question. (3) Learner types or records an answer. (4) Recorded audio is sent through the protected transcription route; transcription text becomes the answer. (5) Service evaluates the answer and returns feedback/score or the next question. (6) Session completes at the selected limit. **Alternatives:** Microphone denial or transcription failure leaves text entry available; service errors are displayed. **Postconditions:** Learner receives session feedback; browser audio is not sent to an external speech provider by this design.

UC-07 — Review Past Tests

Primary actor: Authenticated learner. **Preconditions:** Learner has answered at least one question in a test. **Trigger:** Learner opens My tests. **Main flow:** (1) Backend lists the learner's tests with subject, date, question type, score, average written score and the number of answers flagged as likely AI-written. (2) Learner opens a test. (3) Backend returns each answered question with the learner's answer, the correct option or model answer and marking guide, grade, feedback and AI-writing estimate. **Alternatives:** A test owned by another user is refused; tests taken before history was recorded open without the newer fields. **Postconditions:** No data is changed.

UC-08 — Check Material Authenticity

Primary actor: Authenticated learner. **Preconditions:** AI-writing detector is loaded. **Trigger:** Learner pastes text or uploads a PDF, .txt or .md file. **Main flow:** (1) Backend extracts text (PDFs use the normal extraction and OCR path) and checks it is between 50 and 60,000 characters. (2) ML service scores overlapping 512-token windows and returns the overall and per-window probabilities. (3) Backend saves the result summary and returns the full result. (4) Interface shows the overall band, the section-by-section scores and the learner's recent checks. **Alternatives:** Both inputs, unsupported file types, too-short or too-long text, or a detector that is still loading produce a clear error. **Postconditions:** A result summary is stored; the checked text is not.

3.4 Design Diagrams

3.4.1 Architecture Diagram

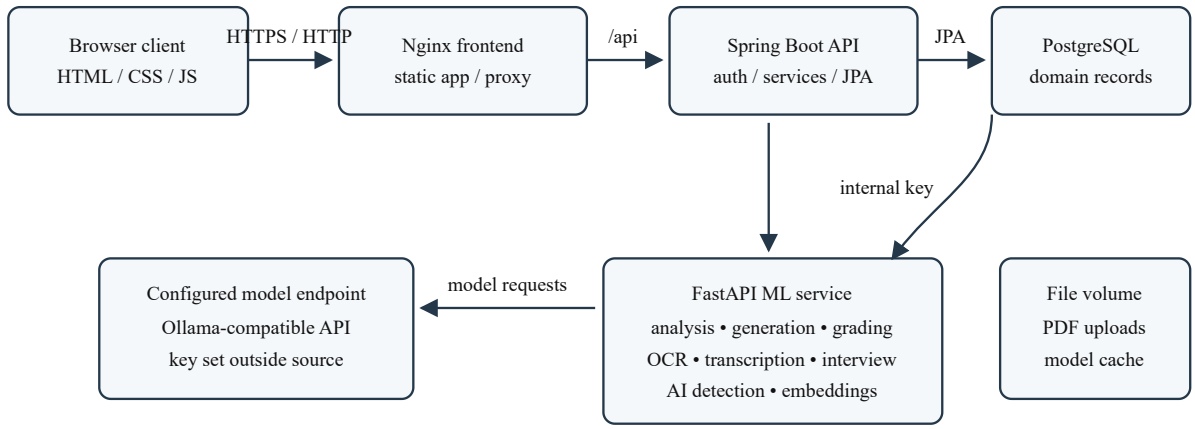


Figure 3.2: Logical deployment architecture. Docker Compose connects the frontend, API, ML service and database; persistent volumes hold uploaded documents and the local model cache.

The browser interacts with the static application and API. Nginx serves frontend assets and routes API traffic according to the deployment configuration. The Spring Boot service owns user-facing authentication, authorization, domain operations and database access. It calls FastAPI using service-to-service configuration for machine-learning tasks. FastAPI performs text clustering and calls the configured language-model endpoint for generation and conversational tasks, and runs the local Whisper, sentence-embedding and AI-writing detector models, which it loads in the background at start-up. PostgreSQL persists relational data, while file storage and the local model cache use persistent Docker volumes. Each container is configured through environment variables; sensitive values must not be committed.

3.4.2 Sequence Diagrams

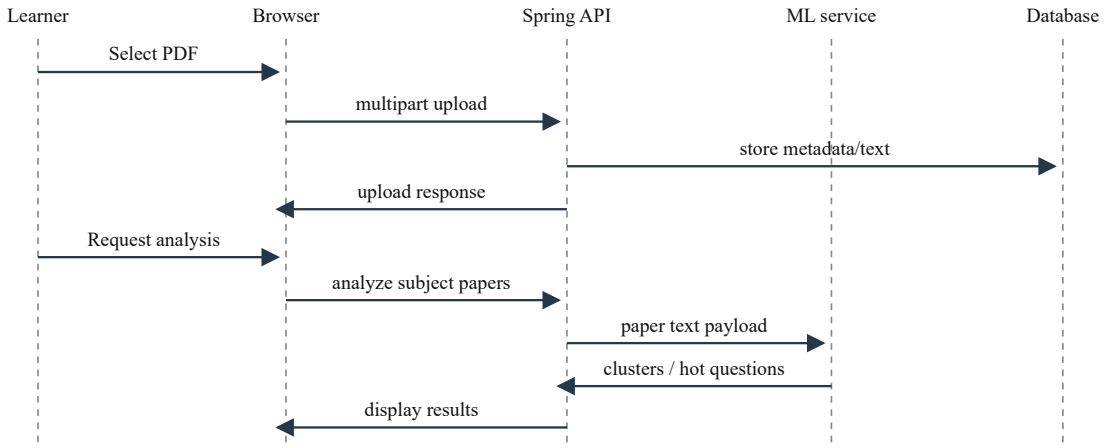


Figure 3.3: Sequence Diagram – Paper Upload and Analysis

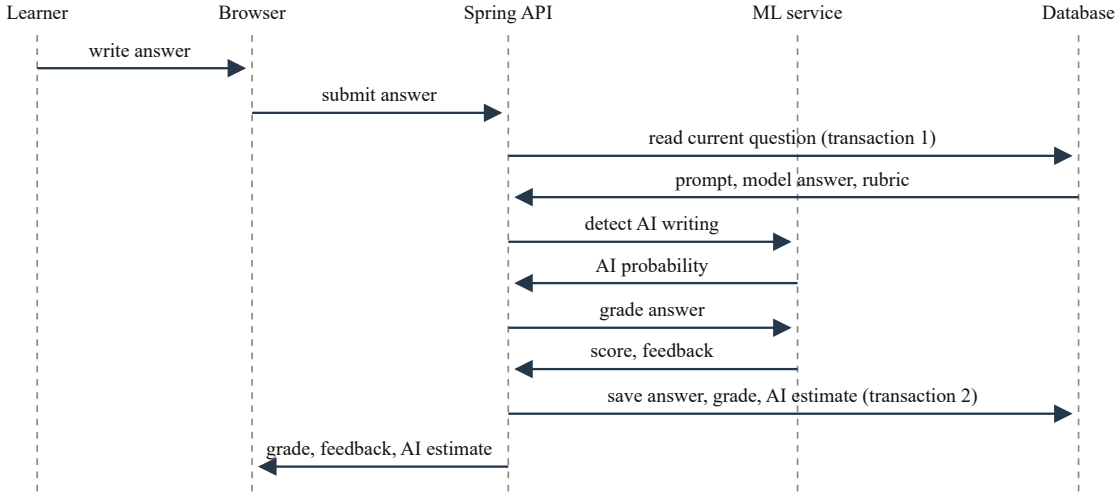


Figure 3.4: Sequence Diagram – Written Answer Grading and AI-Writing Check

CHAPTER 4
IMPLEMENTATION AND TESTING

4.1 Database Design

The backend uses JPA entities mapped to relational tables. The following ER view summarizes the core relations implemented in the domain model. A learner owns subjects; subjects contain topics and uploaded papers; topics contain questions; quiz sessions refer to a learner and subject and maintain a question pool; answers link a session to the question answered and keep the written answer, grade, feedback and AI-writing estimate for later review; authenticity checks belong to a learner. Question options are persisted as an ordered element collection. The many-to-many question pool is represented by a join table.

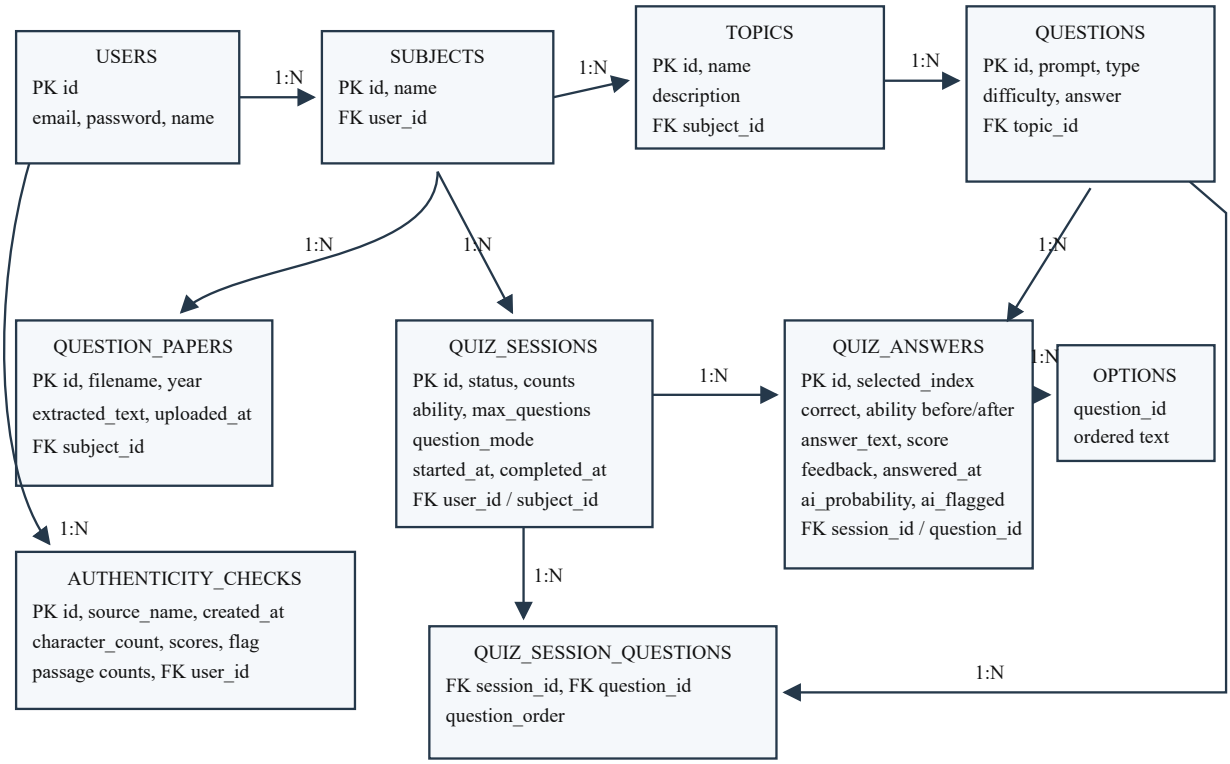


Figure 4.1: Core entity-relationship model. PK denotes primary key; FK denotes foreign key; 1:N denotes one-to-many. QUIZ_SESSION_QUESTIONS resolves the M:N relationship between quiz sessions and questions. AUTHENTICITY_CHECKS stores only the results of material checks.

The structure of each table is shown below, with every field, its data type and constraints, followed by a short description of what the table contains. PK denotes primary key and FK denotes foreign key.

Table 4.1: Structure of the `users` table

Field	Data type	Constraints	Description
`id`	bigint	Primary key, auto-generated (identity)	Unique user identifier
`email`	varchar(255)	Not null, unique	Login email address
`password`	varchar(255)	Not null	BCrypt hash of the password
`full_name`	varchar(255)	Not null	Name shown in the interface

Stores one row per registered learner. The email is the login name and must be unique; the password is never stored in plain text, only as a BCrypt hash.

Table 4.2: Structure of the `subjects` table

Field	Data type	Constraints	Description
`id`	bigint	Primary key, auto-generated (identity)	Unique subject identifier
`name`	varchar(255)	Not null	Subject name, e.g. Operating Systems
`user_id`	bigint	Not null, FK → users.id	Learner who owns the subject

Stores the study subjects that each learner creates. Every subject belongs to exactly one user, and its topics and papers are removed with it.

Table 4.3: Structure of the `topics` table

Field	Data type	Constraints	Description
`id`	bigint	Primary key, auto-generated (identity)	Unique topic identifier
`name`	varchar(255)	Not null	Topic name
`description`	varchar(2000)	Optional	Short description of the topic
`subject_id`	bigint	Not null, FK → subjects.id	Subject the topic belongs to

Stores the topics inside a subject. Generated questions are attached to a topic, so a topic groups the practice material for one part of the syllabus.

Table 4.4: Structure of the `question_papers` table

Field	Data type	Constraints	Description
`id`	bigint	Primary key, auto-generated (identity)	Unique paper identifier

Field	Data type	Constraints	Description
original_filename	varchar(255)	Not null	File name as uploaded by the learner
stored_filename	varchar(255)	Not null, unique	Generated name of the file on disk
exam_year	integer	Optional	Year of the examination
uploaded_at	timestamp	Not null	Date and time of upload
extracted_text	text	Optional	Text extracted from the PDF (directly or by OCR)
subject_id	bigint	Not null, FK → subjects.id	Subject the paper belongs to

Stores one row per uploaded past paper. The PDF file itself is saved on disk under a generated name; this table keeps its metadata and the text extracted from it for analysis and question generation.

Table 4.5: Structure of the `questions` table

Field	Data type	Constraints	Description
id	bigint	Primary key, auto-generated (identity)	Unique question identifier
prompt	varchar(4000)	Not null	Question text
correct_index	integer	Not null	Position of the correct option (multiple choice)
question_type	varchar(32)	MULTIPLE_CHOICE or SUBJECTIVE	Kind of question
reference_answer	varchar(12000)	Optional	Model answer and marking guide (subjective)
explanation	varchar(4000)	Optional	Explanation shown after answering
difficulty	double precision	Not null	Difficulty on the ability scale
times_answered	integer	Not null	How often the question has been answered
topic_id	bigint	Not null, FK → topics.id	Topic the question belongs to

Stores the generated practice questions. A row is either a multiple-choice question (with options and a correct index) or a subjective question (with a model answer and marking guide). The difficulty value is used by the adaptive quiz to pick the next question.

Table 4.6: Structure of the `question_options` table

Field	Data type	Constraints	Description
`question_id`	bigint	Not null, FK → questions.id	Question the option belongs to
`option_order`	integer	Part of primary key	Position of the option (0, 1, 2 …)
`option_text`	varchar(1000)	Not null	Text of the option

Stores the answer options of multiple-choice questions, one row per option, in display order.

Table 4.7: Structure of the `quiz_sessions` table

Field	Data type	Constraints	Description
`id`	bigint	Primary key, auto-generated (identity)	Unique test identifier
`user_id`	bigint	Not null, FK → users.id	Learner taking the test
`subject_id`	bigint	Not null, FK → subjects.id	Subject the test is drawn from
`max_questions`	integer	Not null	Number of questions chosen
`answered_count`	integer	Not null	Questions answered so far
`correct_count`	integer	Not null	Questions answered correctly
`ability_estimate`	double precision	Not null	Current ability estimate of the learner
`status`	varchar(255)	Not null; IN_PROGRESS or COMPLETED	State of the test
`current_question_id`	bigint	Optional, FK → questions.id	Question currently shown
`question_mode`	varchar(32)	Optional	multiple_choice, subjective or mixed
`started_at`	timestamp with time zone	Optional	When the test started
`completed_at`	timestamp with time zone	Optional	When the test finished

Stores one row per test a learner takes. It records the chosen question type and length, the progress so far, the current ability estimate and whether the test is still in progress.

Table 4.8: Structure of the `quiz_session_questions` table

Field	Data type	Constraints	Description
`session_id`	bigint	Not null, FK → quiz_sessions.id	Test the question belongs to
`question_id`	bigint	Not null, FK → questions.id	Question in the pool
`question_order`	integer	Part of primary key	Position of the question in the pool

Join table for the many-to-many relationship between tests and questions. It holds the ordered pool of questions selected for each test.

Table 4.9: Structure of the `quiz_answers` table

Field	Data type	Constraints	Description
`id`	bigint	Primary key, auto-generated (identity)	Unique answer identifier
`session_id`	bigint	Not null, FK → quiz_sessions.id	Test the answer belongs to
`question_id`	bigint	Not null, FK → questions.id	Question that was answered
`selected_index`	integer	Not null	Option chosen (multiple choice)
`correct`	boolean	Not null	Whether the answer was correct
`ability_before`	double precision	Not null	Ability estimate before the answer
`ability_after`	double precision	Not null	Ability estimate after the answer
`answer_text`	varchar(12000)	Optional	Written answer (subjective)
`score`	double precision	Optional	Grade given to a written answer
`feedback`	varchar(4000)	Optional	Feedback on a written answer
`ai_probability`	double precision	Optional	Estimated probability the answer is AI-written
`ai_flagged`	boolean	Optional	Whether the answer was flagged as likely AI-written
`answered_at`	timestamp with time zone	Optional	When the answer was submitted

Stores every answer given in a test. Besides correctness it keeps the ability before and after the answer and, for written answers, the answer text, grade, feedback and AI-writing estimate, so that a test can be reviewed later.

Table 4.10: Structure of the `authenticity_checks` table

Field	Data type	Constraints	Description
`id`	bigint	Primary key, auto-generated (identity)	Unique check identifier
`user_id`	bigint	Not null, FK → users.id	Learner who ran the check
`source_name`	varchar(255)	Not null	File name, or “Pasted text”
`character_count`	integer	Not null	Length of the checked text
`ai_probability`	double precision	Not null	Overall probability of AI-written text
`human_probability`	double precision	Not null	Overall probability of human-written text
`ai_generated`	boolean	Not null	Whether the text was judged likely AI-written
`burstiness_score`	double precision	Not null	Variation in sentence length (supporting signal)
`feedback`	varchar(2000)	Optional	Short explanation of the result
`passage_count`	integer	Not null	Number of sections scored
`flagged_passage_count`	integer	Not null	Sections judged likely AI-written
`created_at`	timestamp with time zone	Not null	When the check was run

Stores the result of each authenticity check a learner runs. Only the result summary is kept; the checked text itself is not stored.

The schema is created/updated by the JPA configuration in the current project. Before production deployment, schema changes should be managed with explicit versioned migrations and retention/backup policies. The columns added for test history are nullable, so tests recorded before the feature remain readable. The database model does not store interview conversation history as a persistent entity in the core domain model shown here.

4.2 Class Diagram

The diagram below shows the central domain and service classes rather than every framework DTO, repository and configuration class in the codebase. It reflects the project’s main responsibility boundaries.

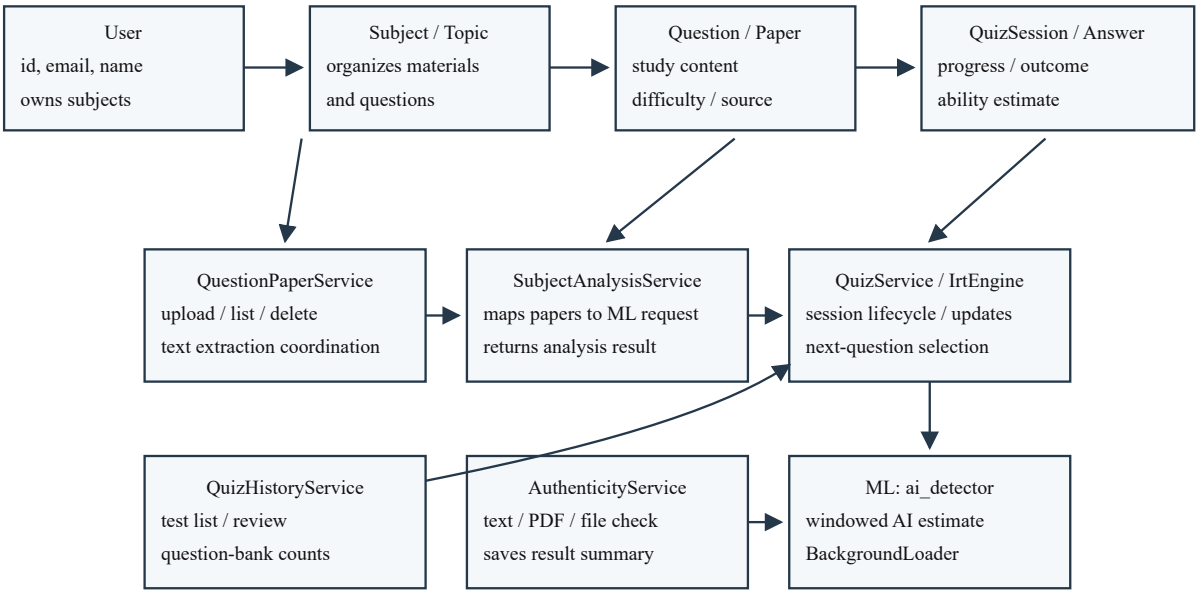


Figure 4.2: Core domain classes and coordinating services. QuizHistoryService reads the sessions and answers recorded by QuizService; written answers and authenticity checks both use the ML service's AI-writing detector.

Class / component	**Purpose and principal behavior**
User	JPA entity for account identity. Holds ID, email, credential value and full name; forms the ownership root for user-specific subjects and quiz sessions.
Subject	JPA entity representing a learner's study area. Associates to an owner and contains topics.
Topic	JPA entity representing a named topic and optional description within a subject.
QuestionPaper	JPA entity holding source filename, stored filename, exam year, upload timestamp, extracted text and owning subject.
Question	JPA entity for a practice prompt, ordered options, answer metadata, question type, explanation, difficulty and topic. It increments the number of answers and exposes question-type-aware accessors.
QuizSession	JPA entity that records learner, subject, maximum question count, progress, ability estimate, current question, status, question pool, chosen question types and start/finish times. `recordAnswer` updates counters/ability; `finish` closes the session and records the completion time.
QuizAnswer	JPA entity linking an answer to its session and question, retaining the selected option, correctness and ability before/after the response; `recordWrittenAnswer` adds the written text, grade, feedback and AI-writing estimate. An earlier version stored the correct option in place of the learner's selection; this was corrected and is covered by a unit test.

Class / component	Purpose and principal behavior
IrtEngine	Stateless utility. `probabilityCorrect(ability, difficulty)` evaluates a logistic response probability; `information` returns p(1-p); `updateAbility` and `updateDifficulty` apply bounded diminishing-step updates; `selectNext` selects the candidate closest to estimated ability.
QuestionPaperService	Coordinates ownership validation, generated filename storage, PDF text extraction, persistence, listing, re-extraction when text is missing and deletion. It cleans up a newly stored file when upload processing fails.
SubjectAnalysisService	Loads a user's subject papers, verifies readable text, maps data to the ML request and returns timestamped analysis metadata.
QuizService	Starts tests, optionally filtering the question pool to multiple-choice or subjective questions, and records answers. For a written answer it reads the question in one short transaction, calls the detector and grader with no transaction open, then stores the result in a second transaction (TransactionTemplate).
QuizHistoryService	Lists a learner's tests with summary scores, builds the per-question review of one test after an ownership check, and counts the subject's multiple-choice and subjective questions for the practice picker.
AuthenticityService / AuthenticityController	Accepts pasted text or a PDF, .txt or .md file, validates its length, calls the detector, saves the result summary and returns the overall and per-section scores; also lists the learner's 20 most recent checks.
ML service (`analyzer`, `embeddings`, `llm_client`, `ai_detector`, `background`)	Python modules split and normalize questions, build TF-IDF and sentence-embedding features, cluster and identify recurring text. The language-model client generates typed questions (with lenient parsing and field mapping), grades subjective answers and manages interview turns. `ai_detector` scores 512-token windows for AI-written text; `BackgroundLoader` loads local models on background threads so requests never wait for a download.

4.3 Test Cases

Testing techniques

Four techniques were used. **Black-box GUI testing** exercised every screen through the browser with valid and invalid inputs chosen by equivalence partitioning (for example a registered and an unregistered email) and boundary values (for example 0, 1 and 6 questions, or text just under 50 characters). **Unit testing** covers deterministic algorithms and service rules, **integration testing** exercises multi-step API and quiz flows against the backend, and the Python tests cover the ML service's analysis, request handling, model client, OCR and transcription. The automated results come from fresh runs on 27 October 2026; the GUI tests were run on 26 October 2026.

Automated test results

Suite / evidence	Test count	Recorded outcome	Evidence note
ML service pytest suite	67	67 passed	Analyzer, API (including internal-key checks and error responses), LLM client (including option relabelling after shuffling), detector, background loading one model at a time, OCR and transcription.
Backend IRT engine	14	14 passed	Ability and difficulty updates; next-question selection.
Backend quiz flow integration	14	14 passed	HTTP-level flows, including question-type practice, stored test review, authenticity checks and server-side validation of registration, subject and topic input.
Backend quiz service	16	16 passed	Quiz rules, type filtering, stored answers and grading without a detector.
Backend interview controller	2	2 passed	Trimming long interview history.
Backend ML client	4	4 passed	Readable ML-service error messages; one retry after a dropped connection.
Backend application context	1	1 passed	Spring context starts.
Backend PDF text extractor	1	1 passed	Embedded-text extraction.
Backend question paper service	1	1 passed	Upload clean-up on failure.

The backend suites total 53 passing tests with no failures or errors (fresh `mvnw clean test` run).

GUI Test Cases

Each screen of the application is shown below with a short description, followed by the test cases run on it: the input given, the output the system displayed, and the result. Every test case that can be seen on screen has a screenshot. All screenshots were taken from the running application (Docker Compose deployment) on 26 and 27 October 2026 with a test account (Riya Verma) and two sample Design and Analysis of Algorithms papers. Error messages from the application appear in a box at the bottom right of the screen. Checks done by the browser (required fields, email format, minimum and maximum values) stop the form before anything is sent; the browser shows its message in a bubble next to the field, which it draws outside the page, so the

screenshots show the rejected input with the field highlighted and the caption gives the browser's exact message.

Registration Page

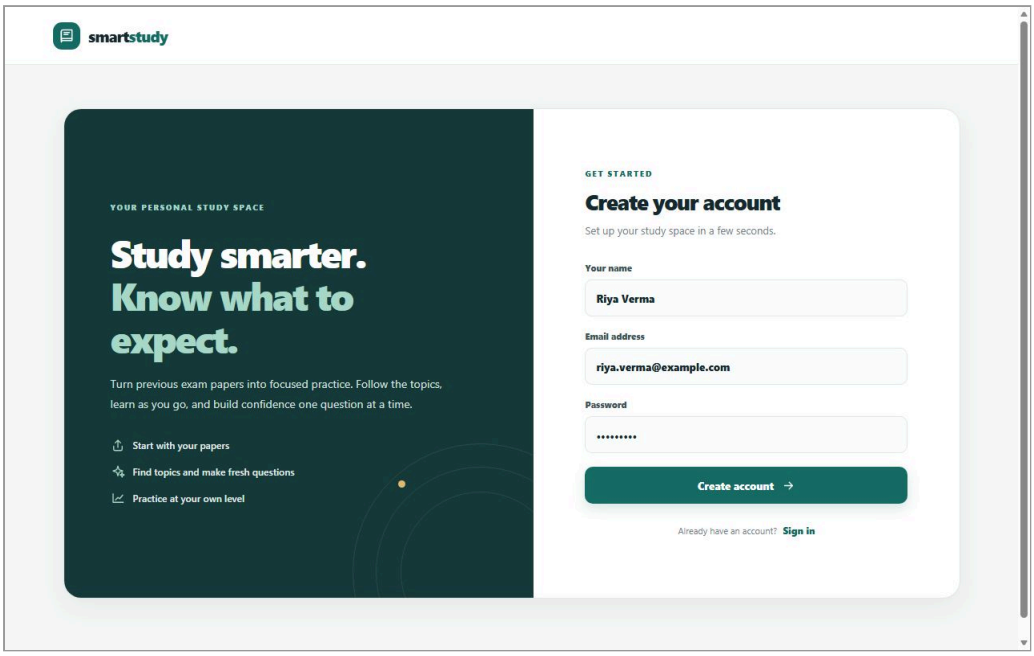


Figure 4.3: Registration page filled in with valid details.

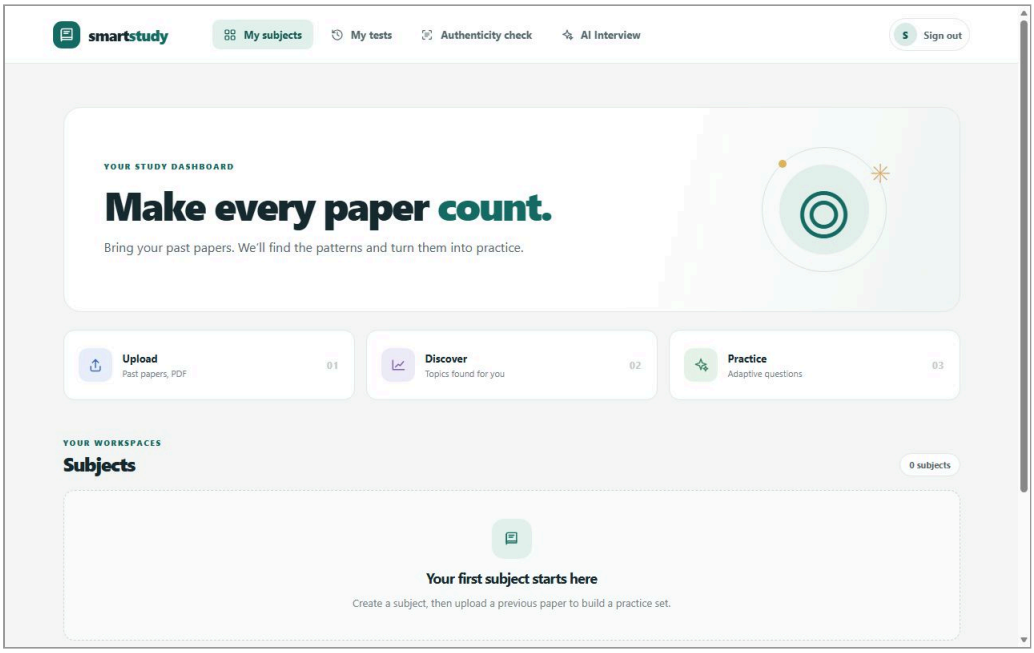


Figure 4.4: Result of a successful registration: the learner is signed in and lands on an empty dashboard.

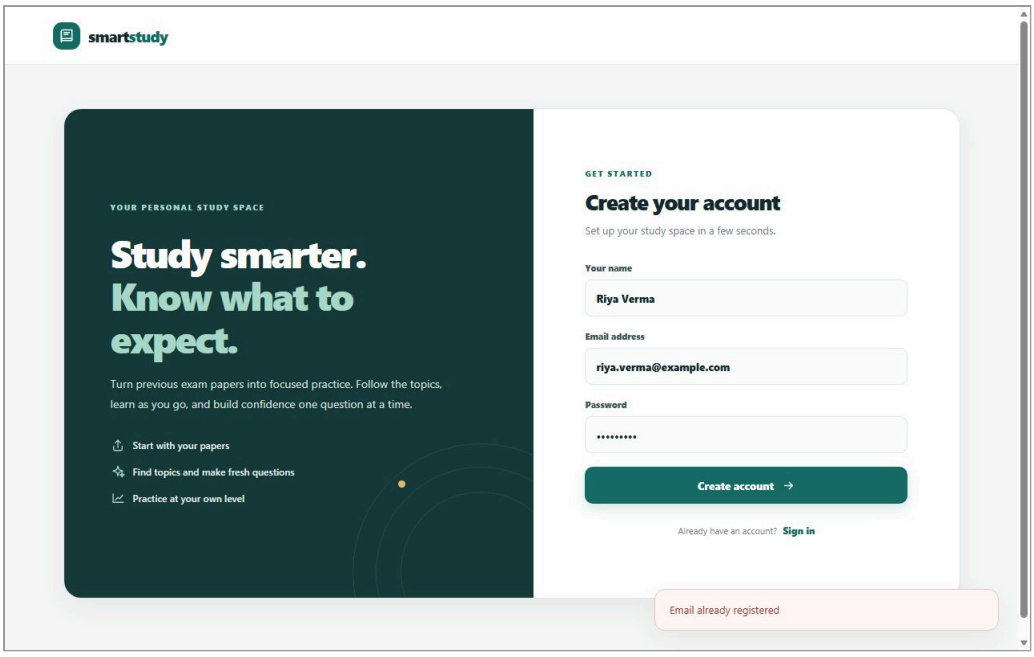


Figure 4.5: Registration with an email address that is already registered: the error message appears at the bottom right.

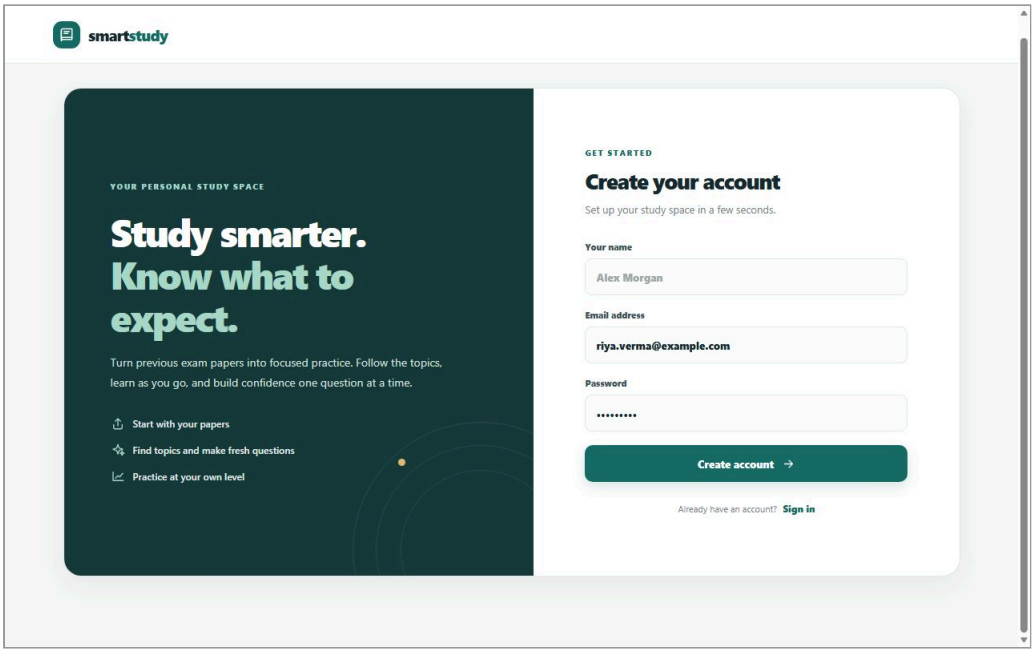


Figure 4.6: Name left empty: the browser blocks the form and shows “Please fill out this field.” next to the field.

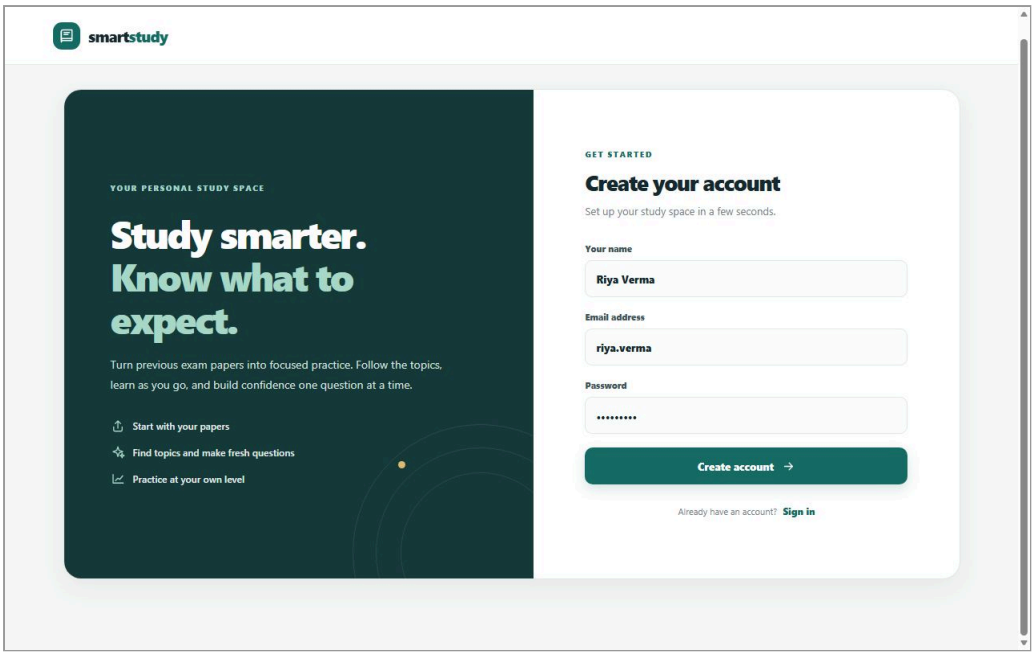


Figure 4.7: Email address without “@”: the browser blocks the form and shows “Please include an '@' in the email address. 'riya.verma' is missing an '@'.” next to the field.

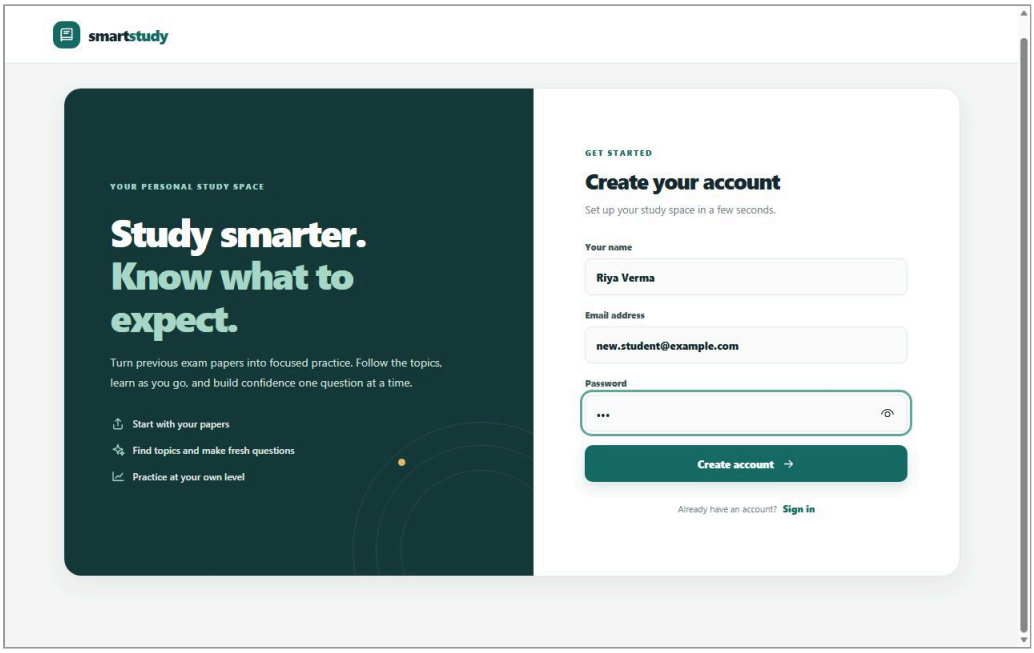


Figure 4.8: Password of 3 characters: the browser blocks the form and shows “Please lengthen this text to 6 characters or more (you are currently using 3 characters).” next to the field.

The registration page collects the learner’s name, email address and password. On success the account is created, the learner is signed in automatically and the dashboard opens. The browser checks for required fields, the email format and a minimum password length of six characters before sending the form, and the server checks the same rules again and rejects an email address that already has an account.

Test	Test case	Input	System output	Result
Test 01	Successful registration	Name: Riya Verma Email: riya.verma@example.com Password: Study@123	Account created; learner signed in and taken to the dashboard (Figures 4.3 and 4.4).	Pass
Test 02	Email already registered	Name: Riya Verma Email: riya.verma@example.com Password: Study@123	Error message “Email already registered” (Figure 4.5); no second account is created.	Pass
Test 03	Name left empty	Name: (empty) Email: riya.verma@example.com Password: Study@123	Browser blocks the form and shows “Please fill out this field.”; nothing is sent to the server (Figure 4.6).	Pass
Test 04	Invalid email address	Name: Riya Verma Email: riya.verma Password: Study@123	Browser blocks the form and shows “Please include an '@' in the email address. 'riya.verma' is missing an '@'.”; nothing is sent to the server (Figure 4.7).	Pass
Test 05	Password shorter than 6 characters	Name: Riya Verma Email: new.student@example.com Password: abc	Browser blocks the form and shows “Please lengthen this text to 6 characters or more (you are currently using 3 characters).”; nothing is sent to the server (Figure 4.8).	Pass

Sign-in Page

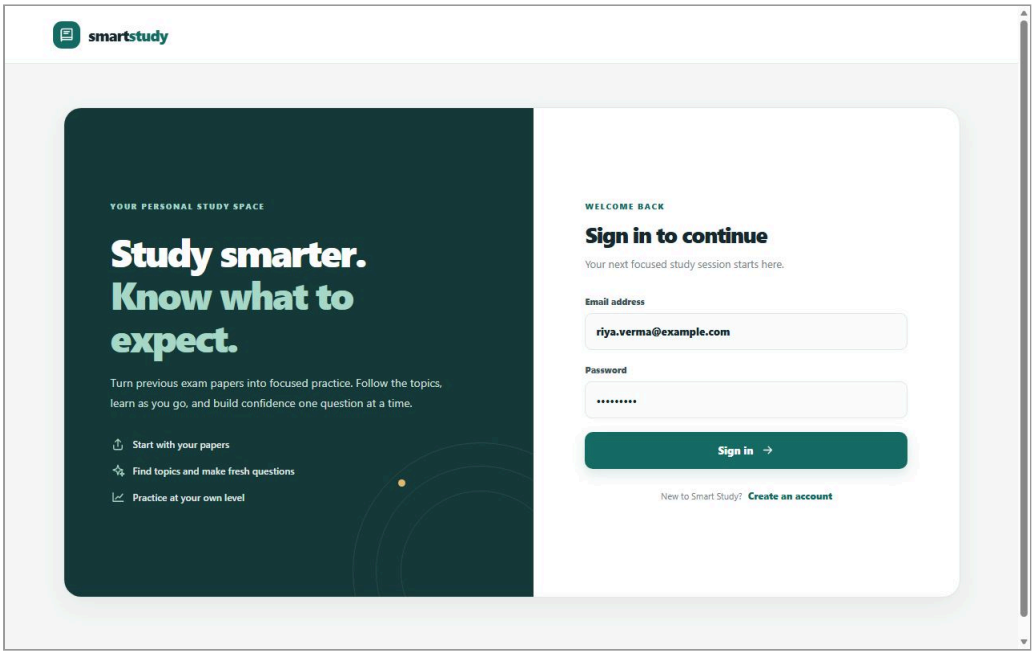


Figure 4.9: Sign-in page filled in with a registered email address and its password.

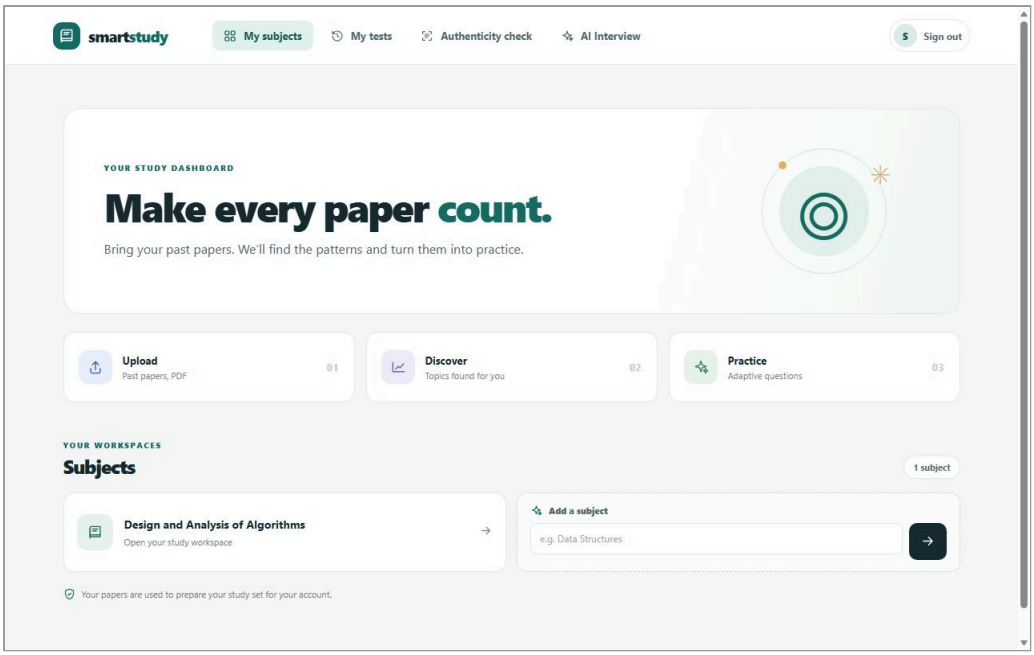


Figure 4.10: Result of a successful sign-in: the dashboard with the learner's subject.

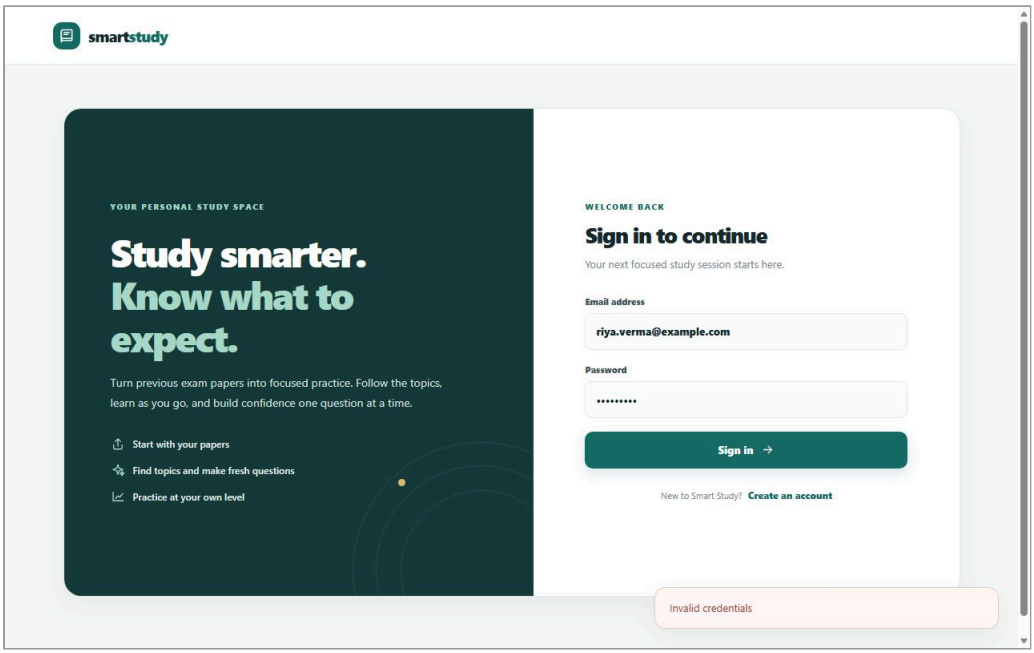


Figure 4.11: Sign-in with an incorrect password: the error message appears at the bottom right.

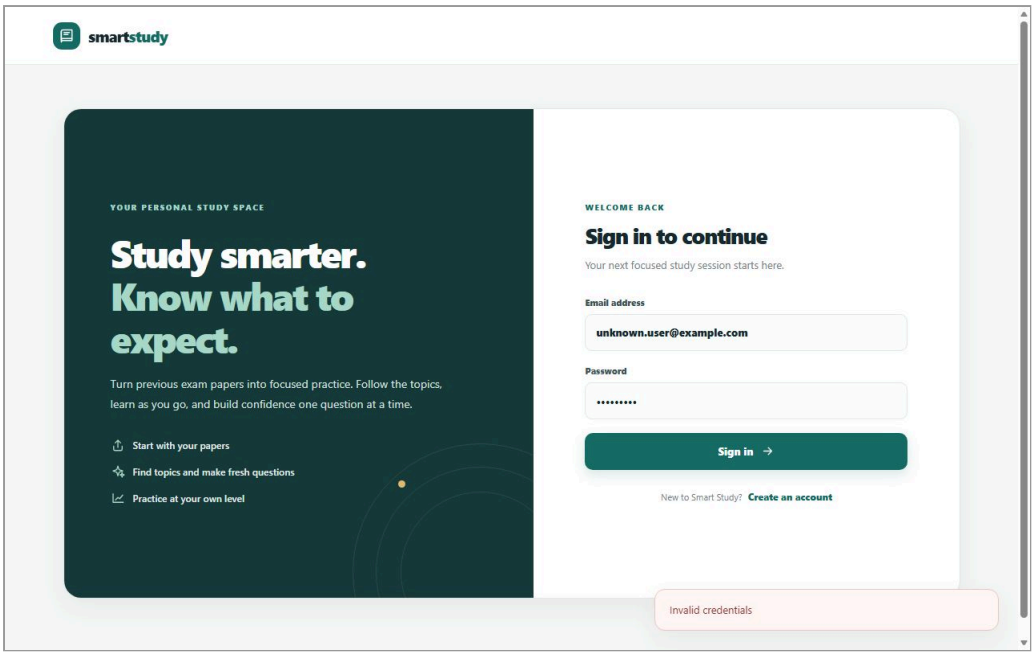


Figure 4.12: Sign-in with an email address that has no account: the same error message appears.

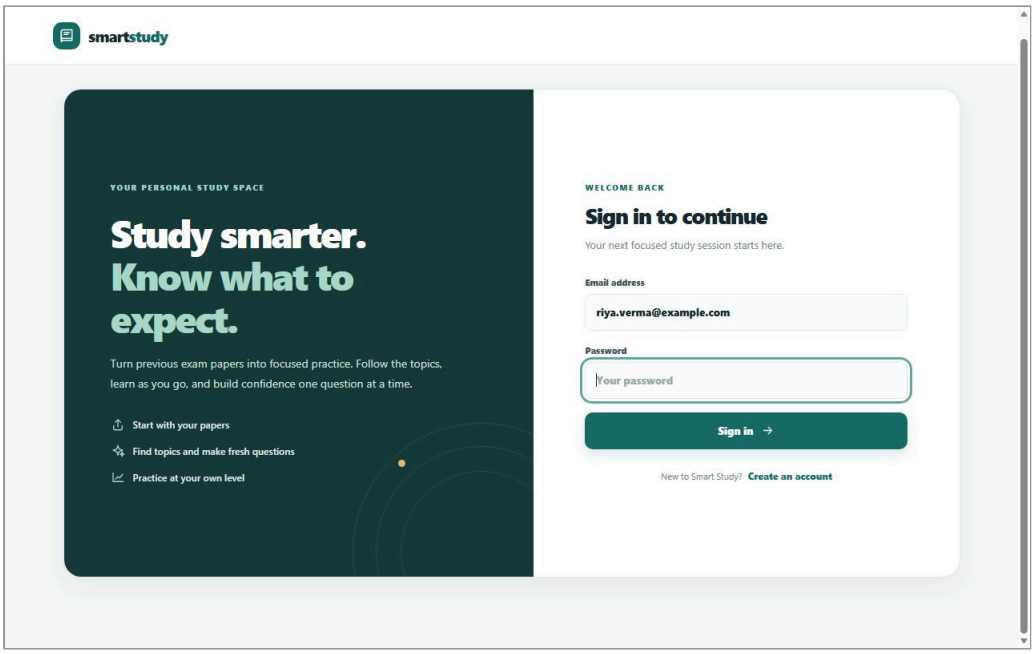


Figure 4.13: Password left empty: the browser blocks the form and shows “Please fill out this field.” next to the field.

Registered learners sign in with their email address and password. A successful sign-in opens the dashboard; an incorrect password or an unknown email address gives the same message, so the page does not reveal which email addresses have accounts.

Test	Test case	Input	System output	Result
Test 06	Successful sign-in	Email: riya.verma@example.com Password: Study@123	Learner signed in and taken to the dashboard (Figures 4.9 and 4.10).	Pass

Test	Test case	Input	System output	Result
Test 07	Incorrect password	Email: riya.verma@example.com Password: Study@999	Error message “Invalid credentials” (Figure 4.11).	Pass
Test 08	Email not registered	Email: unknown.user@example.com Password: Study@123	Error message “Invalid credentials” (Figure 4.12).	Pass
Test 09	Password left empty	Email: riya.verma@example.com Password: (empty)	Browser blocks the form and shows “Please fill out this field.”; nothing is sent to the server (Figure 4.13).	Pass

Dashboard and Add Subject

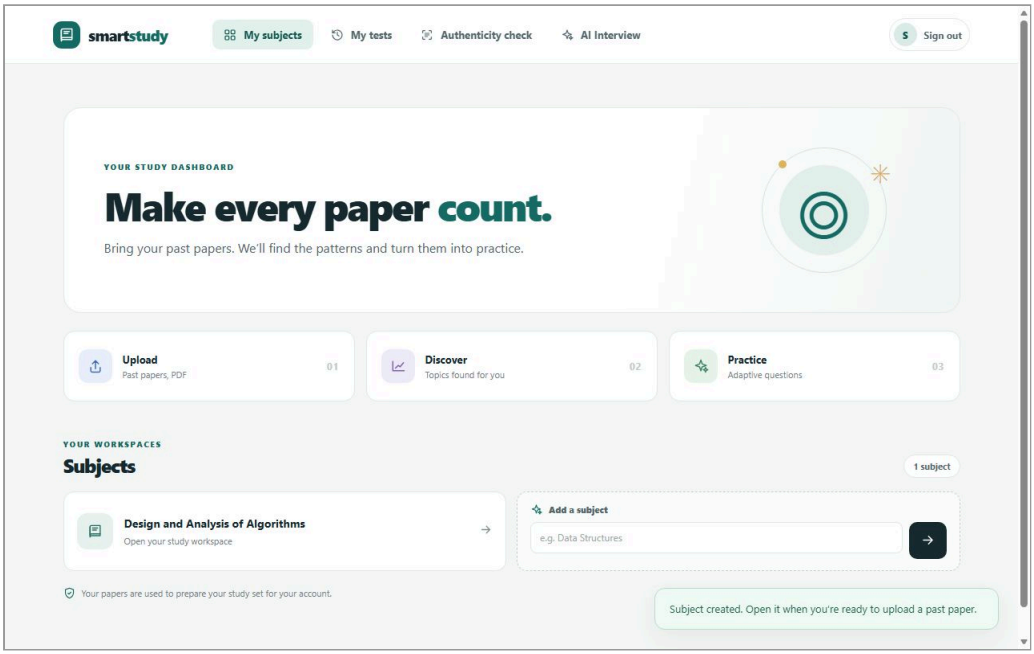


Figure 4.14: Dashboard after adding a subject: the new subject card and the confirmation message.

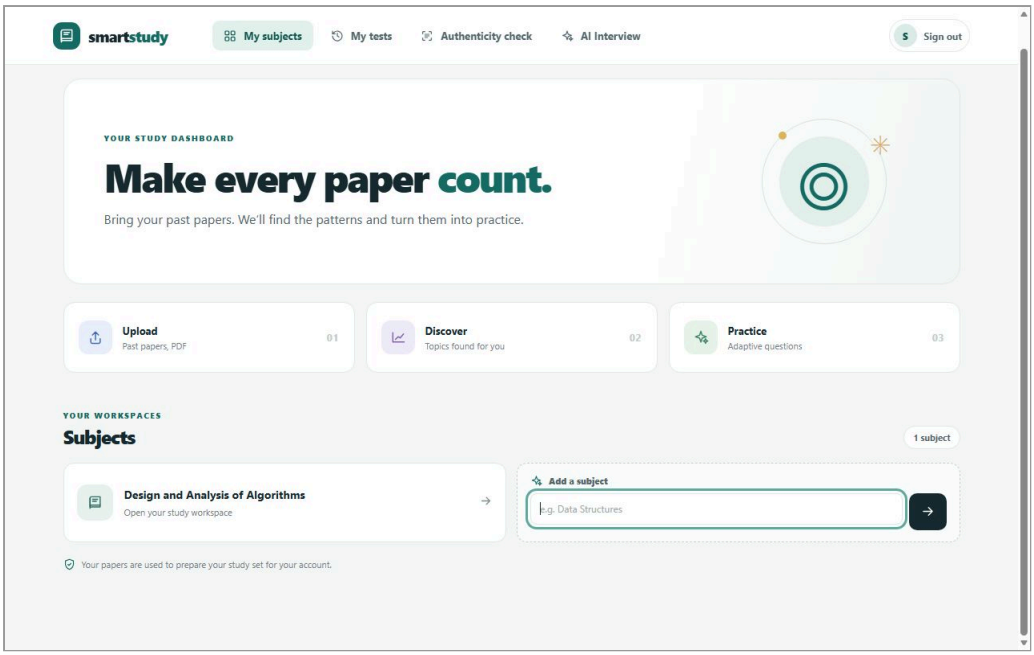


Figure 4.15: Subject name left empty: the browser blocks the form and shows “Please fill out this field.” next to the field.

The dashboard lists the learner’s subjects and has a form to add a new one. After a subject is added its card appears in the list and a confirmation message is shown. Selecting a card opens the subject page.

Test	Test case	Input	System output	Result
Test 10	Add a subject	Subject name: Design and Analysis of Algorithms	Subject card added; message “Subject created. Open it when you’re ready to upload a past paper.” (Figure 4.14).	Pass
Test 11	Subject name left empty	Subject name: (empty)	Browser blocks the form and shows “Please fill out this field.”; nothing is sent to the server (Figure 4.15).	Pass

Build a Practice Set (Upload Past Papers and Generate Questions)

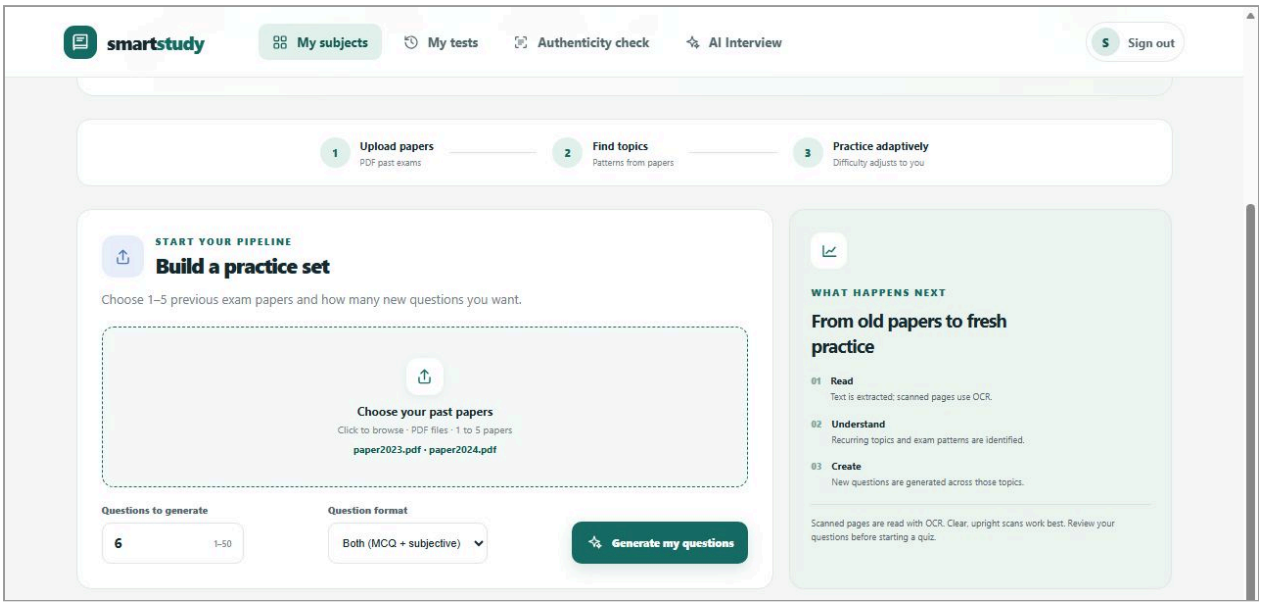


Figure 4.16: Subject page: two past papers chosen, 6 questions requested in both formats.

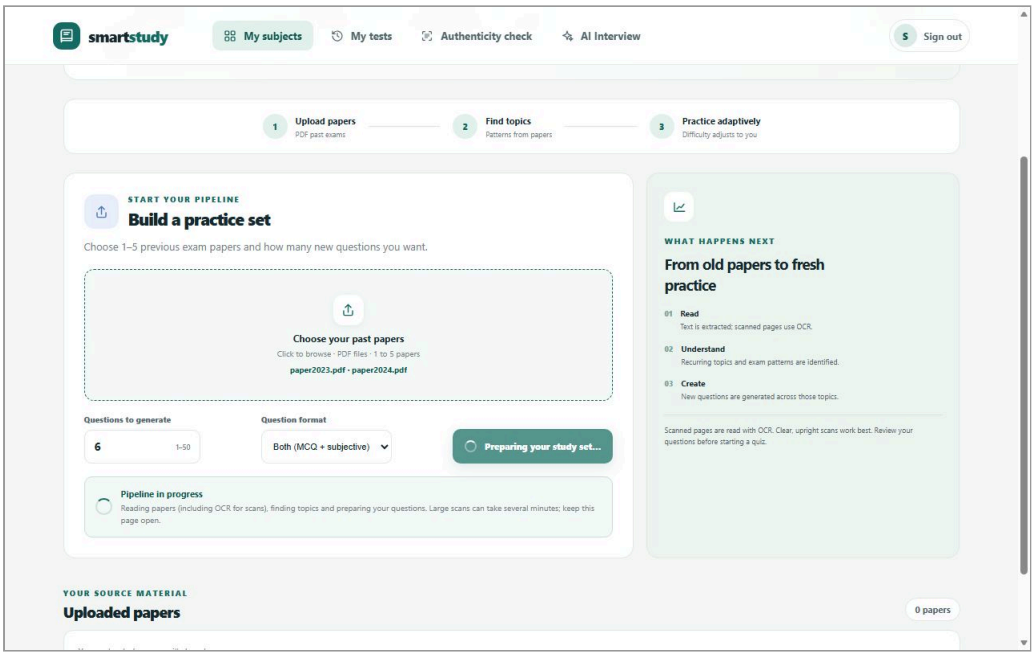


Figure 4.17: Processing in progress after selecting Generate my questions.

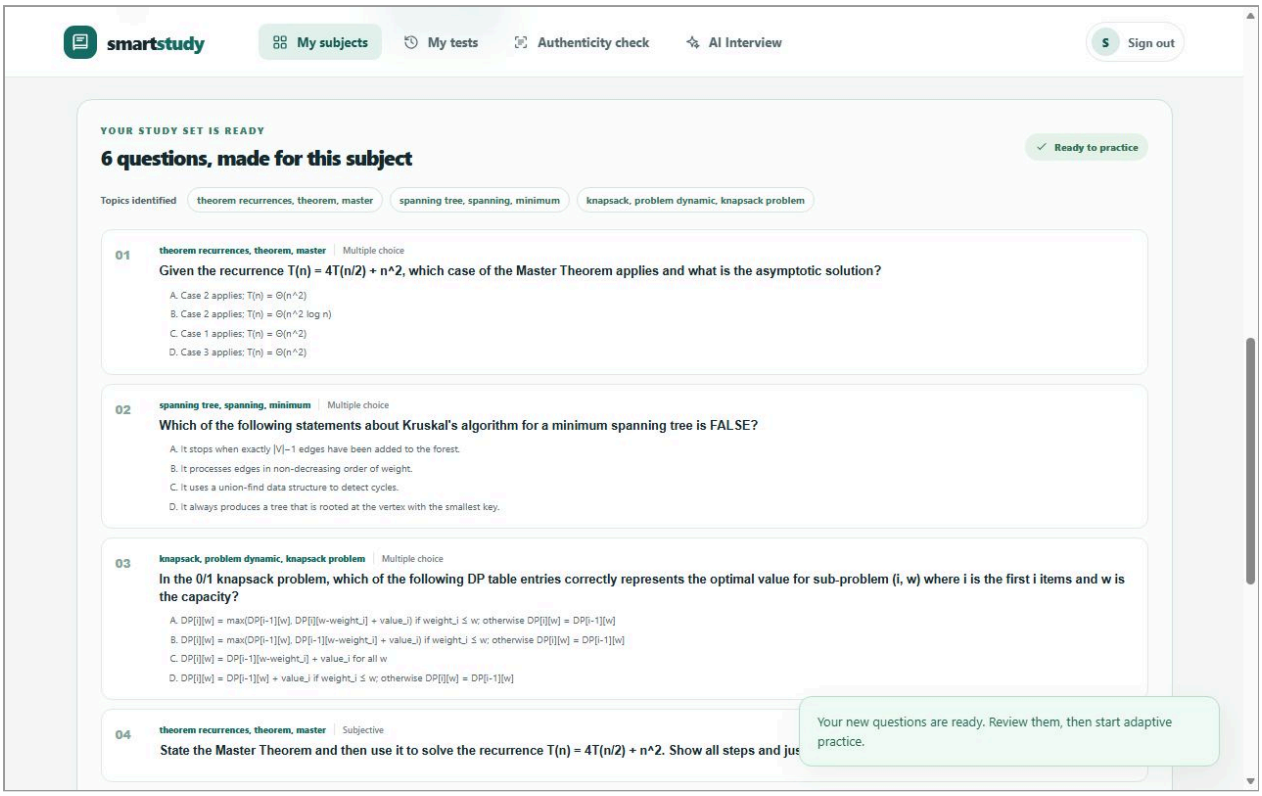


Figure 4.18: Generated practice set: 6 questions with the topics identified in the papers.

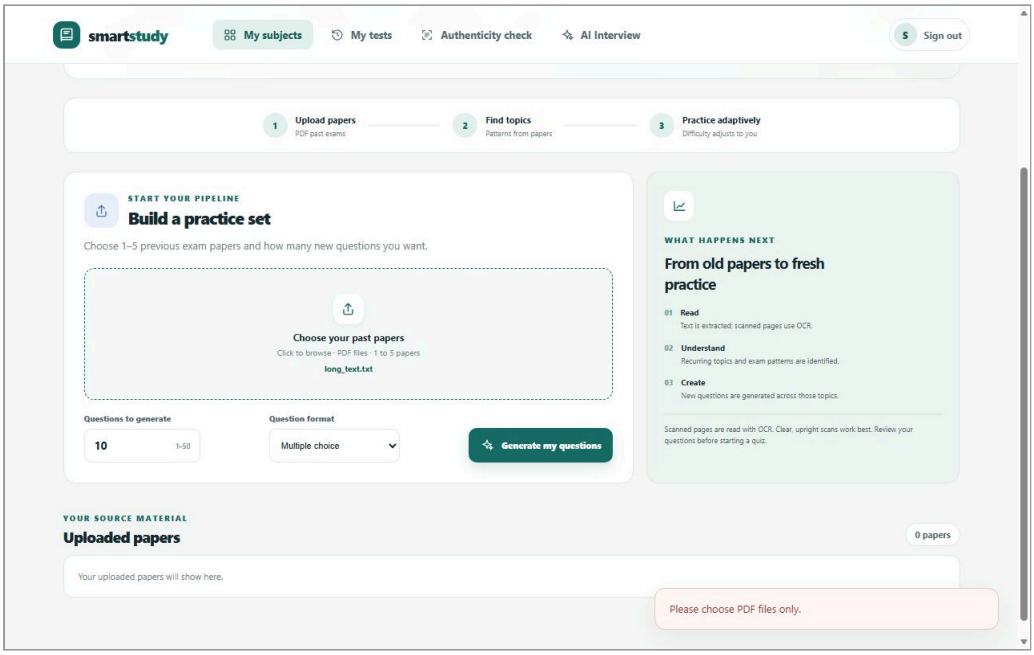


Figure 4.19: A text file chosen instead of a PDF: the error message appears at the bottom right.

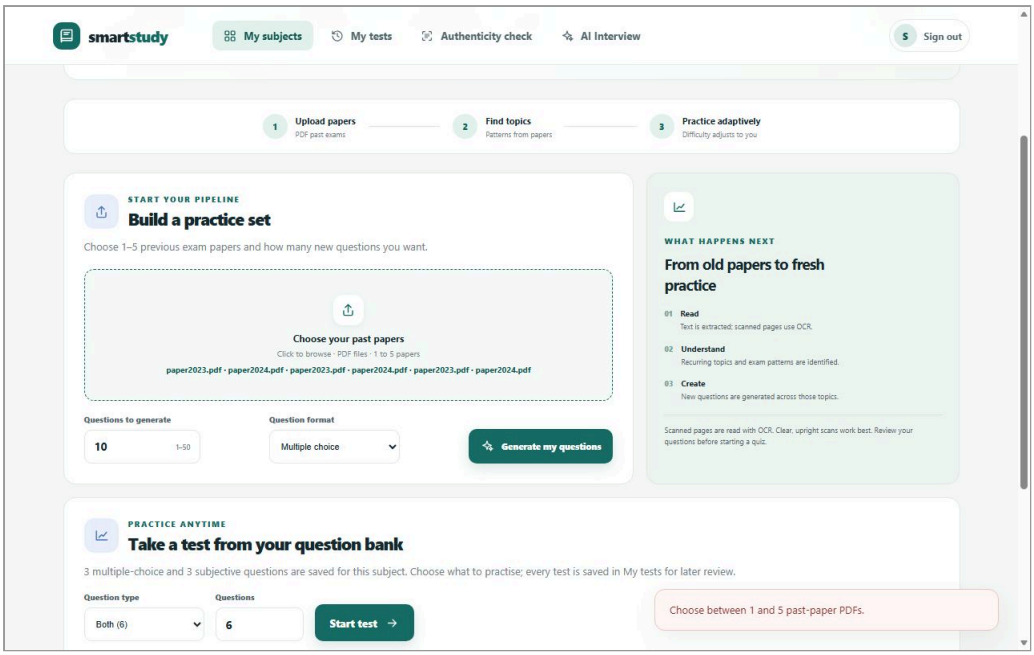


Figure 4.20: Six PDF files chosen: the error message appears at the bottom right.

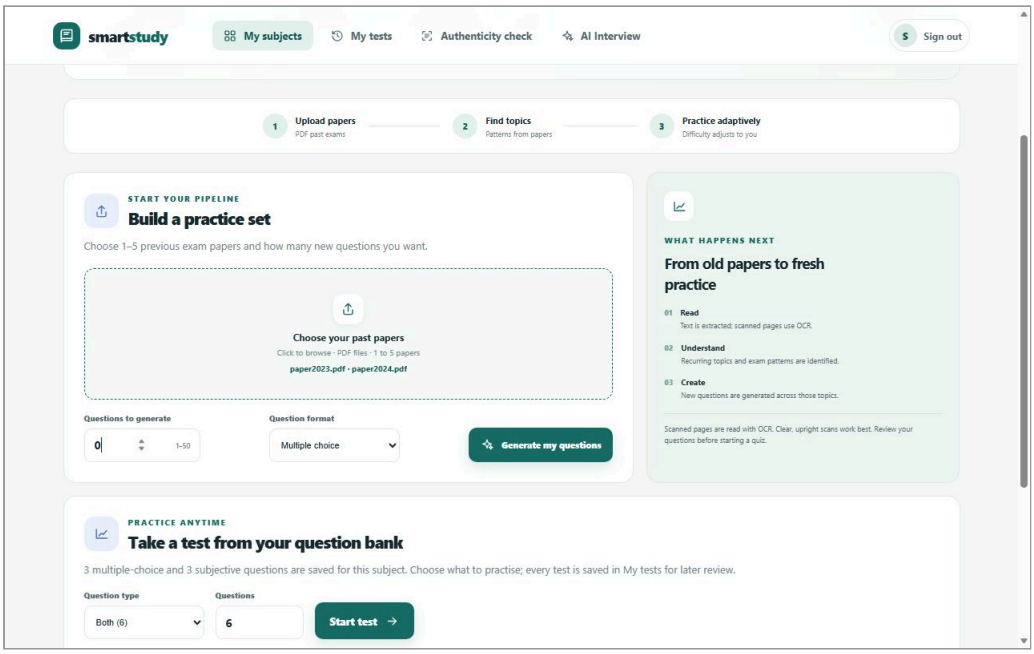


Figure 4.21: Questions to generate set to 0: the browser blocks the form and shows “Value must be greater than or equal to 1.” next to the field.

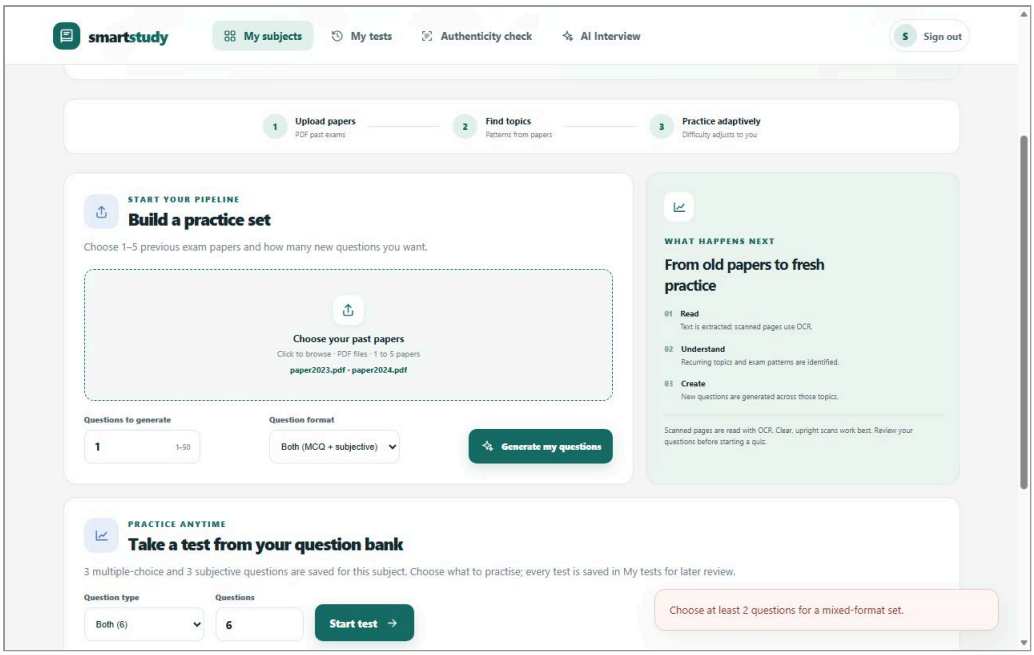


Figure 4.22: One question requested in both formats: the error message appears at the bottom right.

On the subject page the learner chooses one to five past-paper PDFs, the number of questions and the question format, then selects *Generate my questions*. The papers are read, recurring topics are identified and new questions are generated across those topics. Two sample papers (Design and Analysis of Algorithms, 2023 and 2024) were used; the system identified three topics (Master theorem recurrences, minimum spanning trees and the knapsack problem) and generated one multiple-choice and one subjective question for each.

Test	Test case	Input	System output	Result
Test 12	Generate a mixed practice set	Files: paper2023.pdf, paper2024.pdf Questions: 6 Format: Both	Progress panel shown, then “6 questions, made for this subject” with three topics and the message “Your new questions are ready. Review them, then start adaptive practice.” (Figures 4.16, 4.17 and 4.18).	Pass
Test 13	File is not a PDF	File: long_text.txt	Error message “Please choose PDF files only.” (Figure 4.19).	Pass
Test 14	More than 5 files	Files: 6 PDF files	Error message “Choose between 1 and 5 past-paper PDFs.” (Figure 4.20).	Pass
Test 15	Question count below the minimum	Files: 2 PDFs Questions: 0	Browser blocks the form and shows “Value must be greater than or equal to 1.”; nothing is sent to the server (Figure 4.21).	Pass
Test 16	Too few questions for both formats	Files: 2 PDFs Questions: 1 Format: Both	Error message “Choose at least 2 questions for a mixed-format set.” (Figure 4.22).	Pass

Adaptive Multiple-Choice Test

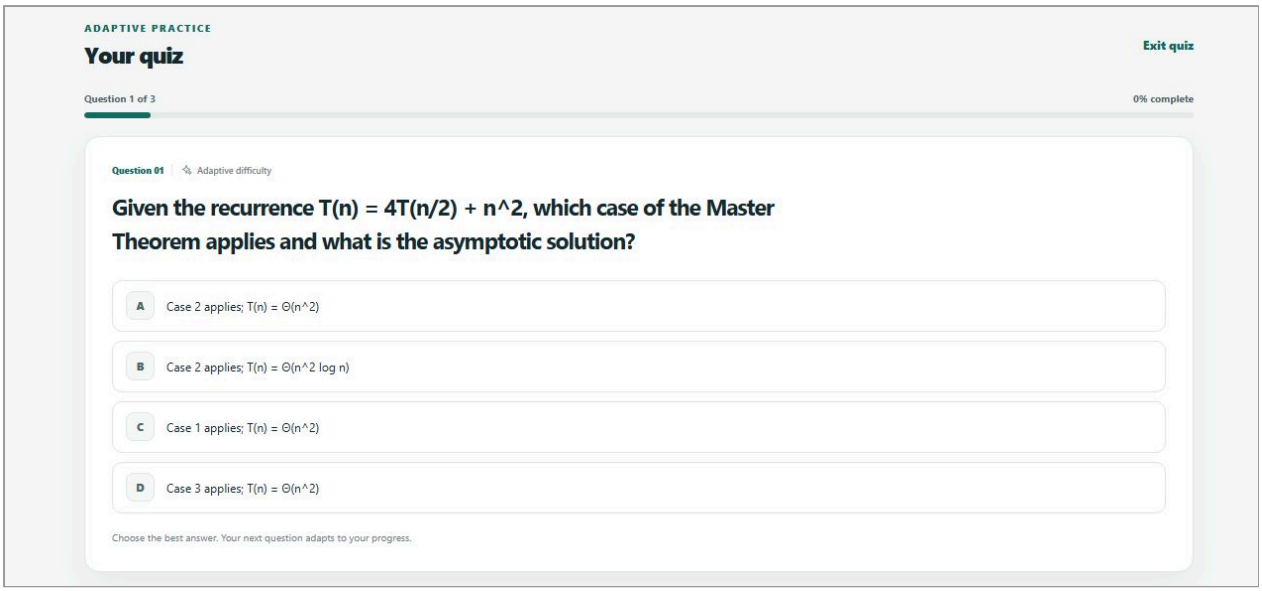


Figure 4.23: A multiple-choice question in an adaptive test.

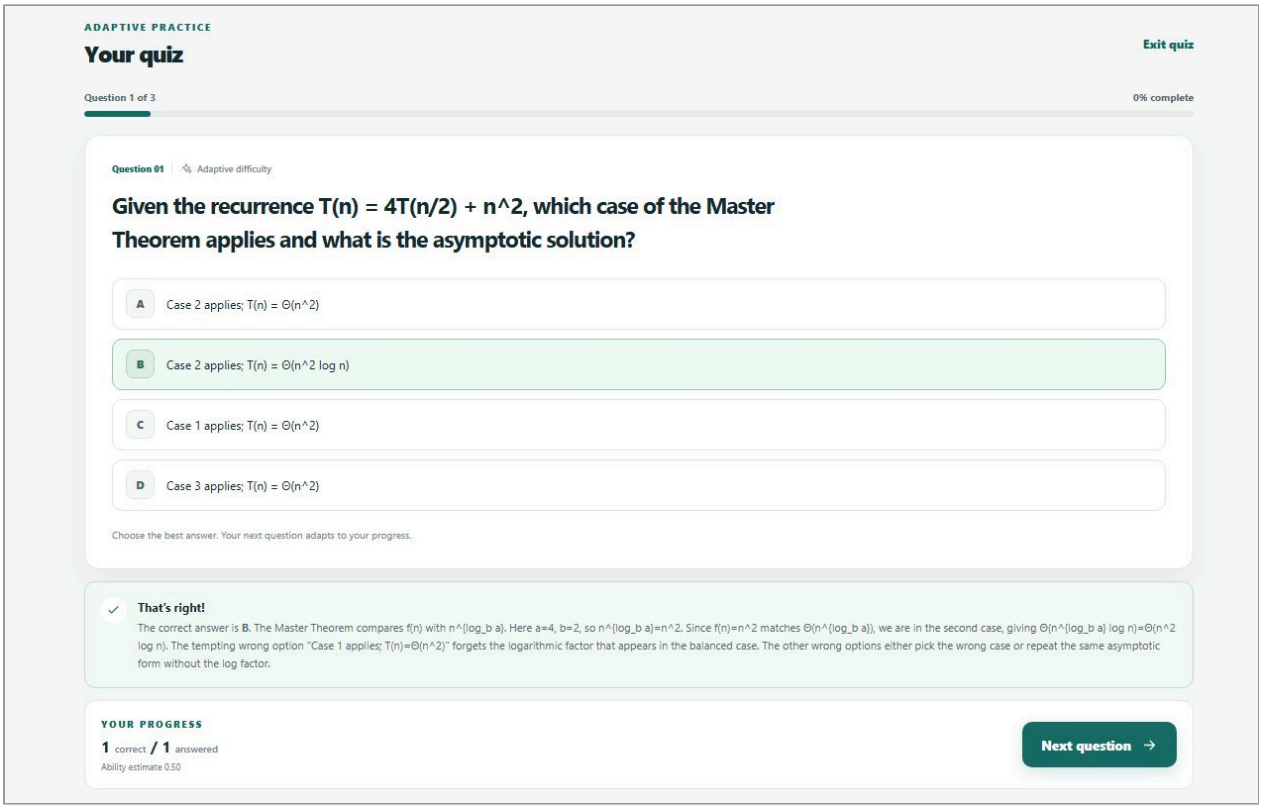


Figure 4.24: Feedback after choosing the correct option.

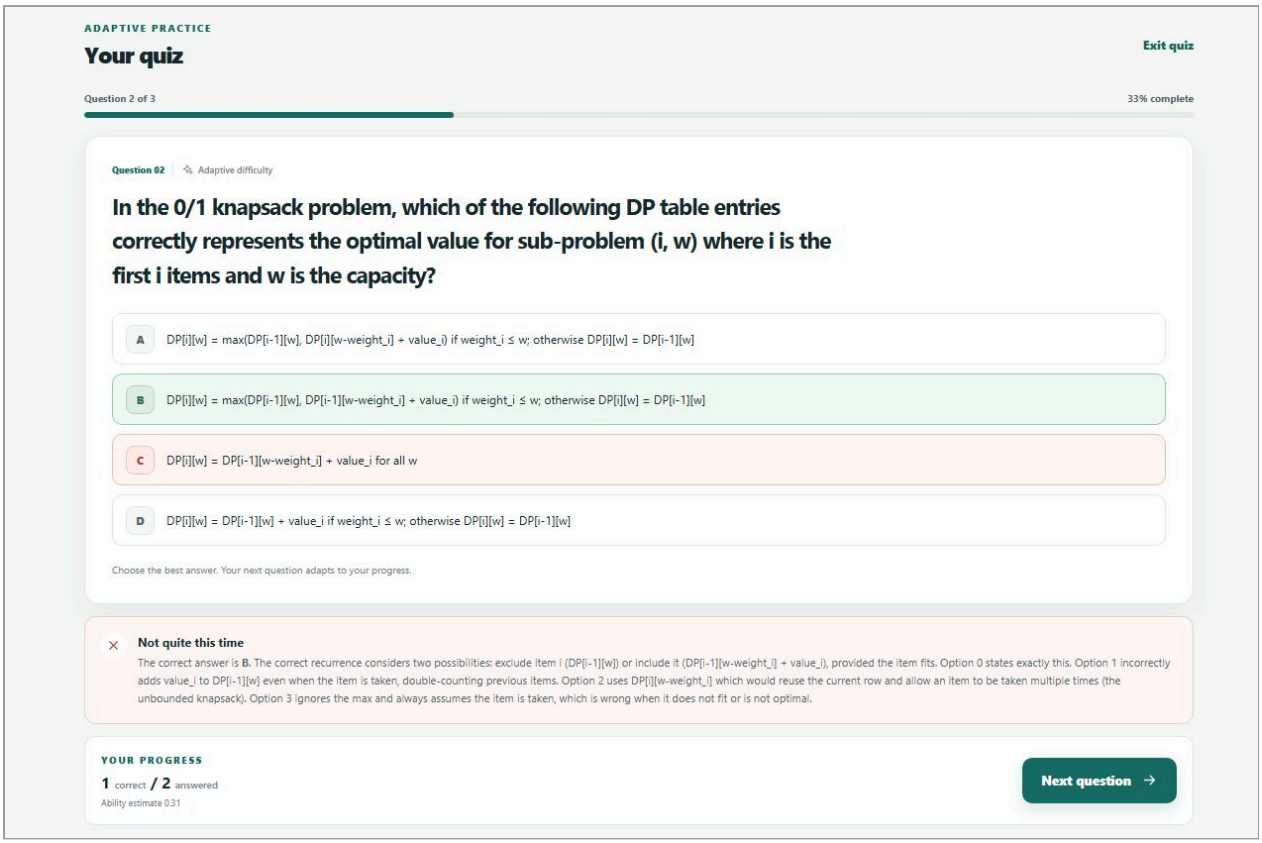


Figure 4.25: Feedback after choosing an incorrect option: the chosen option is marked red and the correct one green.

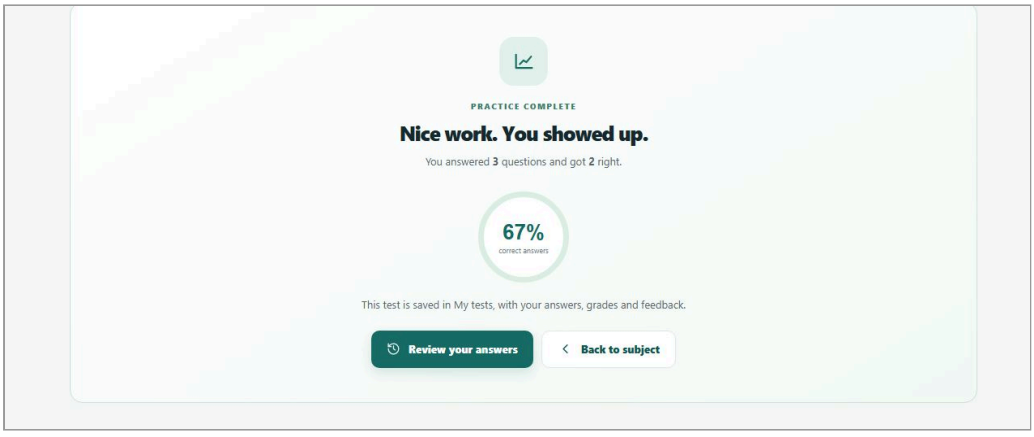


Figure 4.26: Result at the end of the test.

Under *Take a test from your question bank* the learner chooses the question type and number of questions. Each answer is marked immediately with the correct option and an explanation, and the ability estimate is updated; the next question is chosen with a difficulty close to that estimate. A test of three multiple-choice questions was taken, answering the second one incorrectly on purpose.

Test	Test case	Input	System output	Result
Test 17	Start a multiple-choice test	Question type: Multiple choice Questions: 3	Test starts with question 1 of 3 and four options (Figure 4.23).	Pass
Test 18	Choose the correct option	Option B for the Master theorem question	“That’s right!”, explanation shown; ability estimate rises from 0.00 to 0.50 (Figure 4.24).	Pass
Test 19	Choose an incorrect option	Option C for the knapsack question	“Not quite this time”, correct answer B highlighted; ability estimate falls from 0.50 to 0.31 (Figure 4.25).	Pass
Test 20	Finish the test	Answer question 3 correctly, then select See your results	“You answered 3 questions and got 2 right”, 67% correct; test saved in My tests (Figure 4.26).	Pass

Written (Subjective) Answer with Grading and AI-Writing Check

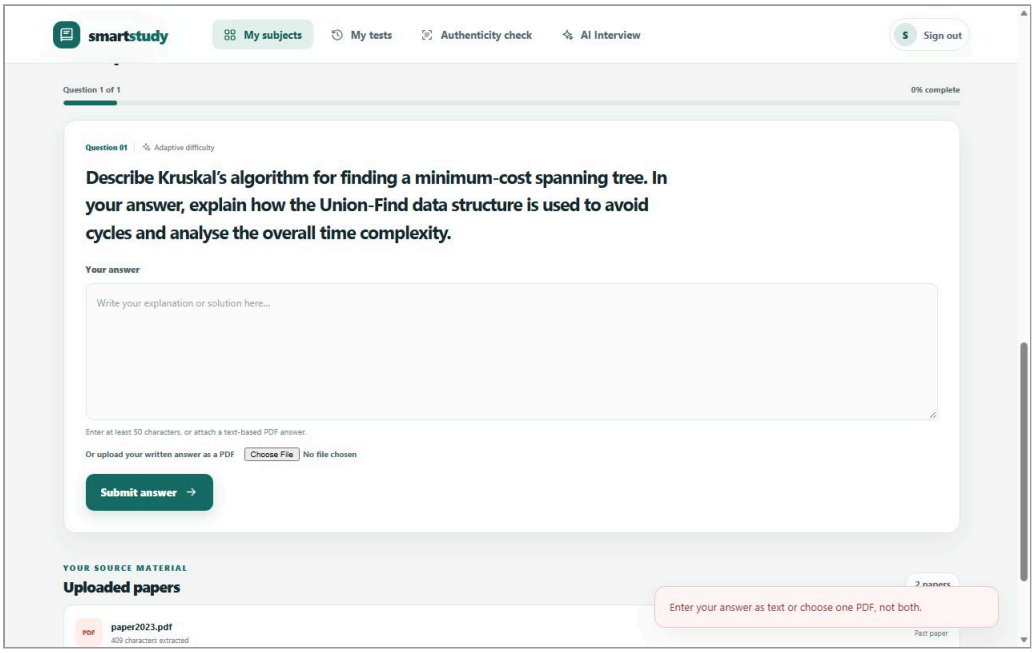


Figure 4.27: A written answer submitted with nothing entered: the error message appears at the bottom right.

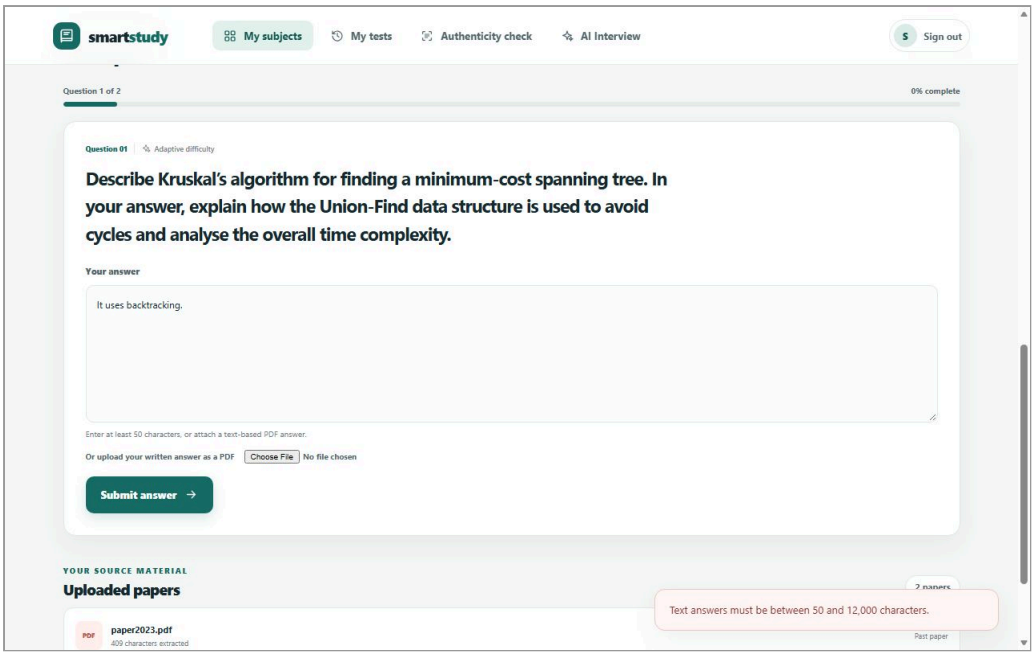


Figure 4.28: A written answer that is too short: the error message appears at the bottom right.

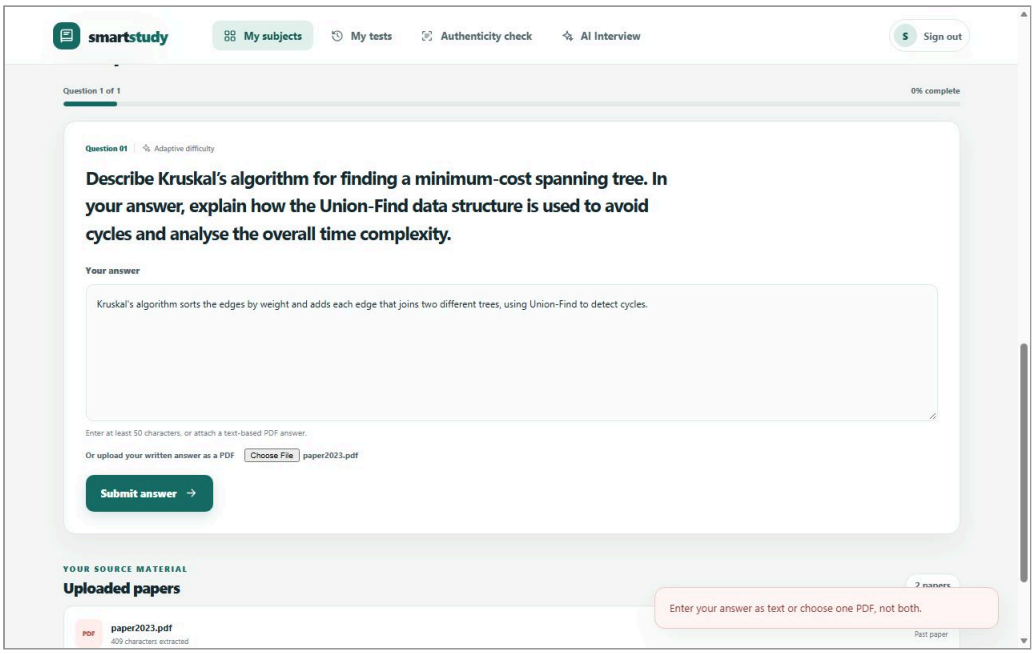


Figure 4.29: Typed text and a PDF submitted together: the error message appears at the bottom right.

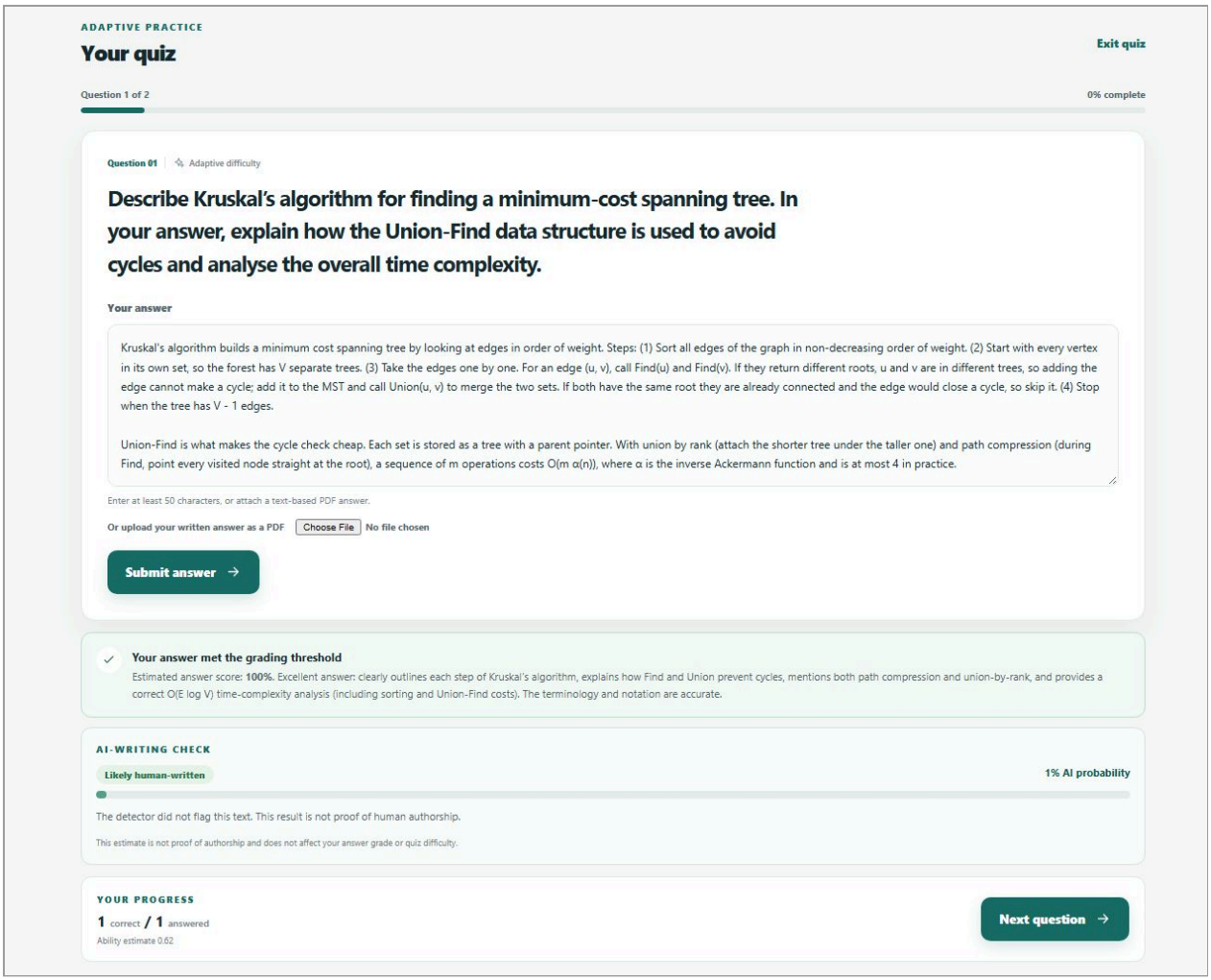


Figure 4.30: A complete written answer: grade, feedback and AI-writing check.

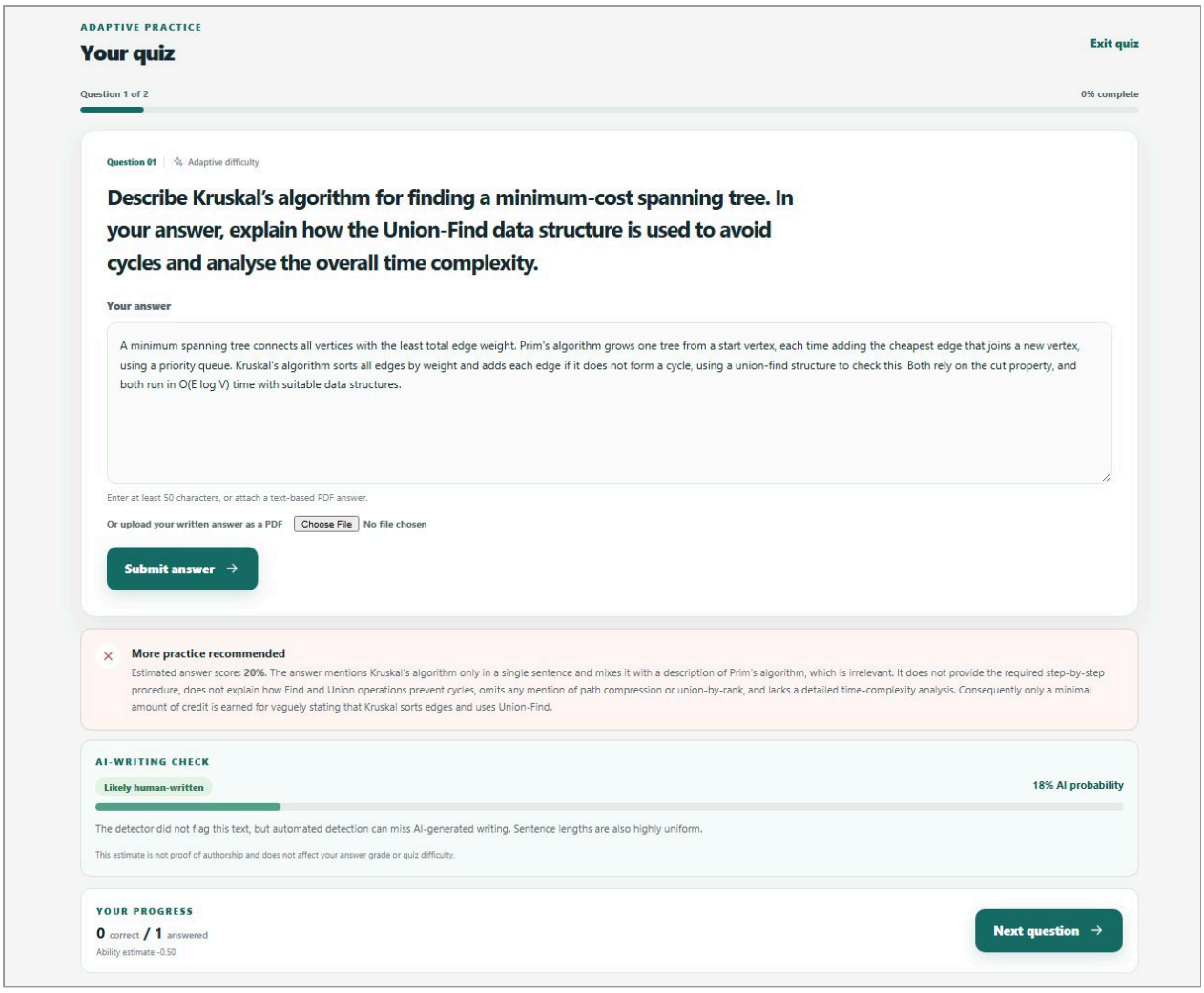


Figure 4.31: A brief, partly off-topic written answer: low grade with feedback on what is missing.

For a subjective question the learner types an answer (or uploads it as a PDF) and selects *Submit answer*. The answer is graded against the model answer and marking guide, feedback is shown, and a separate AI-writing check estimates how likely it is that the answer was AI-written. The estimate never changes the grade.

Test	Test case	Input	System output	Result
Test 21	Submit an empty answer	Answer: (empty), no PDF	Error message “Enter your answer as text or choose one PDF, not both.” (Figure 4.27).	Pass
Test 22	Answer shorter than 50 characters	Answer: “It uses backtracking.”	Error message “Text answers must be between 50 and 12,000 characters.” (Figure 4.28).	Pass
Test 23	Both text and a PDF	Answer: typed text PDF: paper2023.pdf	Error message “Enter your answer as text or choose one PDF, not both.” (Figure 4.29).	Pass

Test	Test case	Input	System output	Result
Test 24	Complete, correct answer	Step-by-step answer on Kruskal's algorithm, Union-Find and O(E log V) complexity	"Your answer met the grading threshold", score 100%, feedback; AI-writing check: likely human-written, 1% (Figure 4.30).	Pass
Test 25	Brief, partly off-topic answer	Short paragraph mixing Prim's and Kruskal's algorithms	"More practice recommended", score 20%, with feedback listing the missing points (Figure 4.31).	Pass

My Tests and Test Review

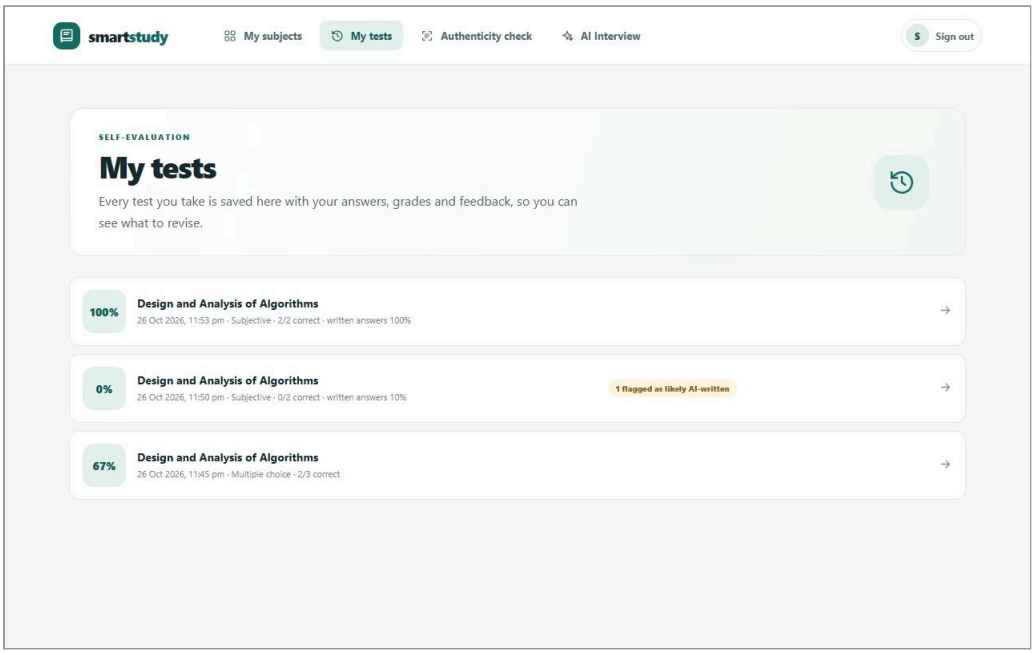


Figure 4.32: My tests: every test with its date, question type and score.

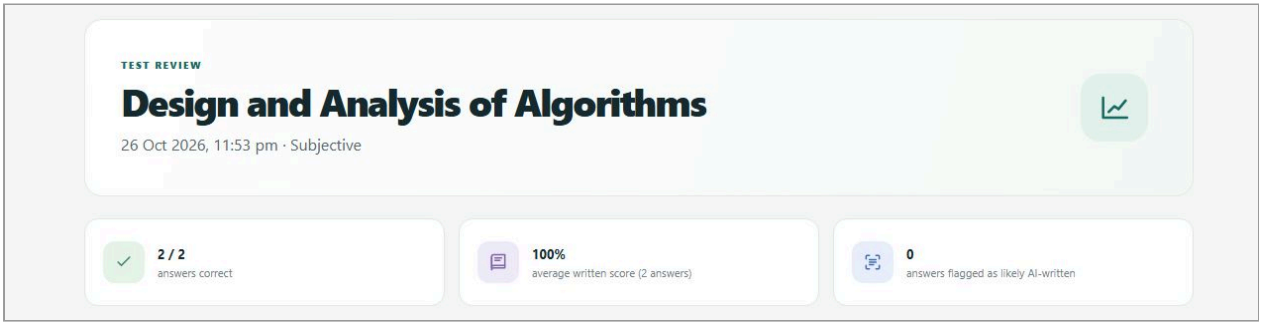


Figure 4.33: Test review summary: answers correct, average written score and answers flagged as likely AI-written.

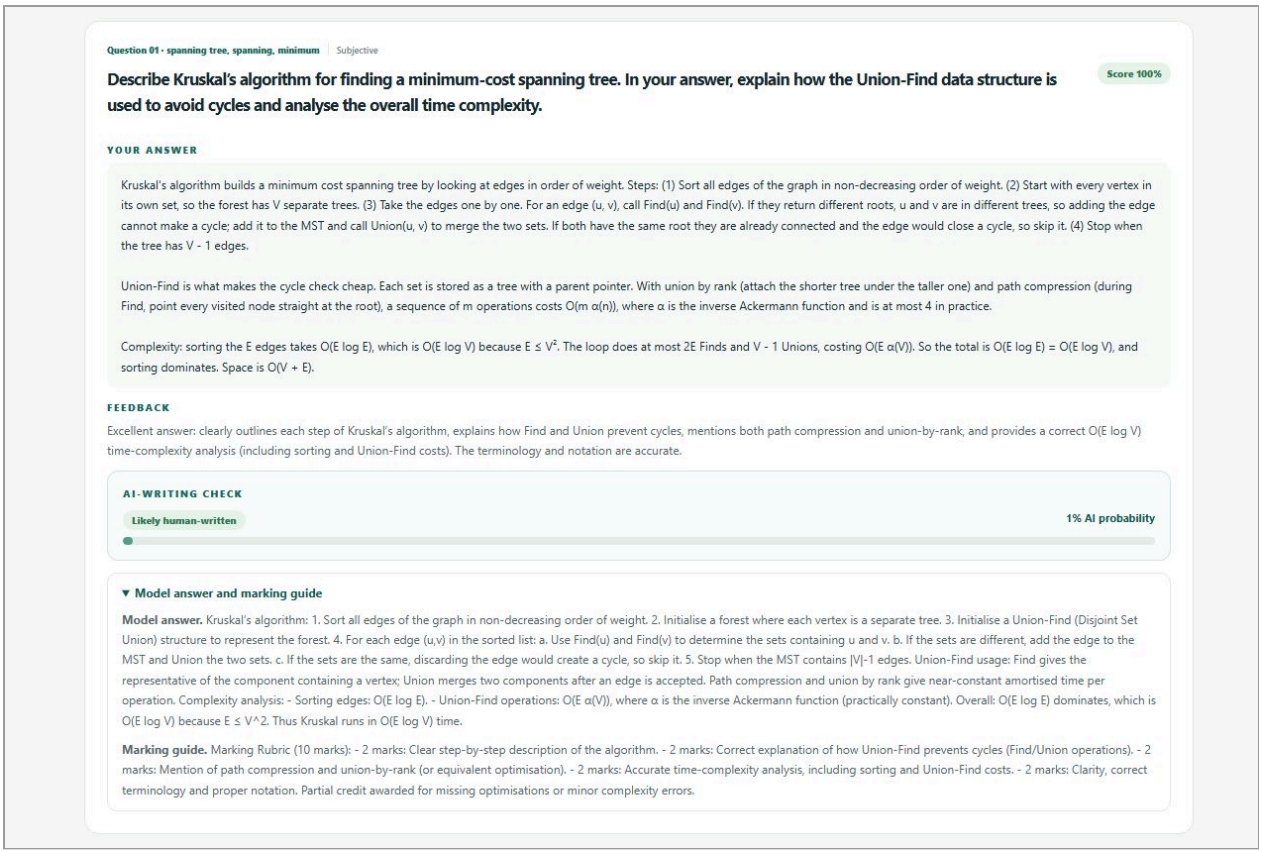


Figure 4.34: Test review of one written answer: the answer, feedback, AI-writing check and the model answer with marking guide.

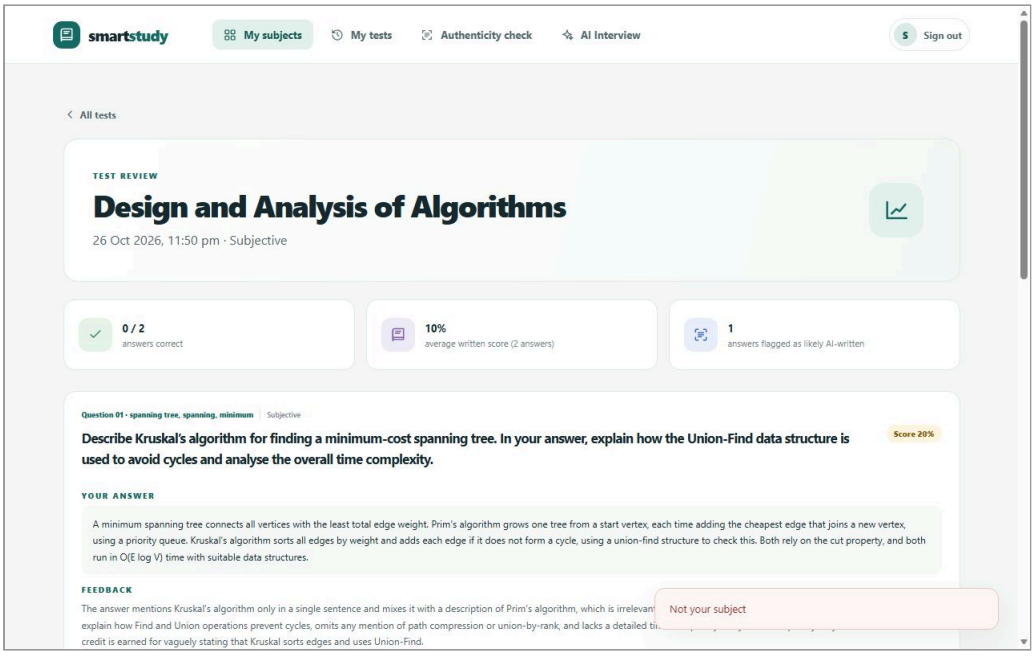


Figure 4.35: Opening another learner's subject by its address: access is refused.

Every test is saved. *My tests* lists them with date, question type and score, and selecting one shows each question with the learner's answer, the correct option or model answer, the grade,

feedback and AI-writing check. A learner can see only their own data: typing the address of another learner's subject is refused.

Test	Test case	Input	System output	Result
Test 26	Open My tests	Select My tests in the navigation bar	Three tests listed with date, type and score: 100% (subjective), 0% (subjective, 1 flagged), 67% (multiple choice) (Figure 4.32).	Pass
Test 27	Review a test	Select the 100% subjective test	2/2 correct, average written score 100%, 0 flagged; each answer with feedback and model answer (Figures 4.33 and 4.34).	Pass
Test 28	Open another learner's subject	Address: #/subjects/1 (owned by a different account)	Error message "Not your subject"; the subject is not shown (Figure 4.35).	Pass

Authenticity Check

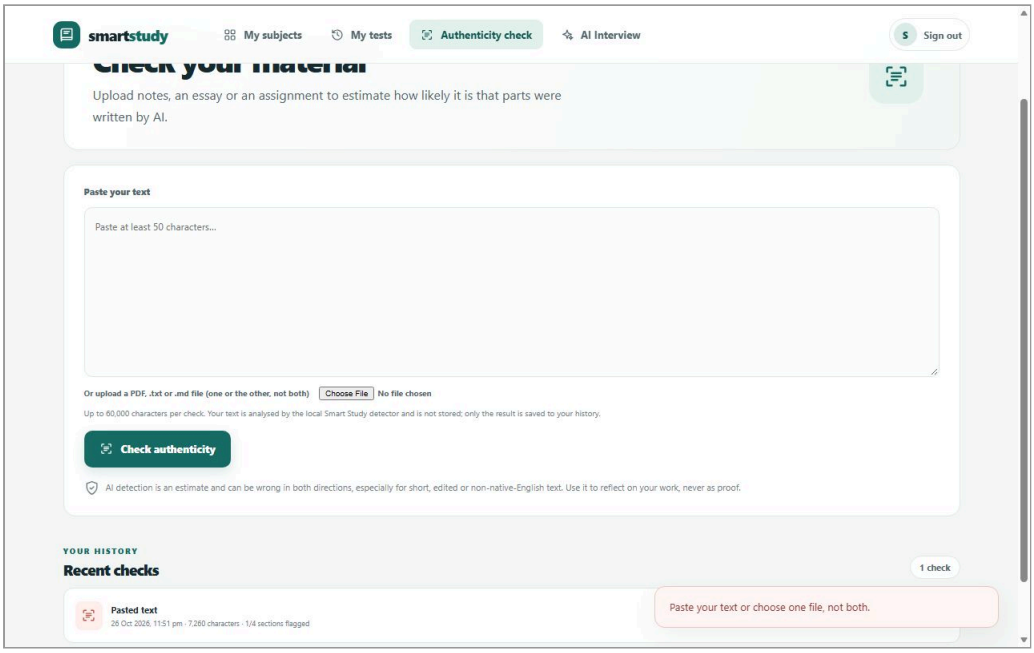


Figure 4.36: Check selected with nothing entered: the error message appears at the bottom right.

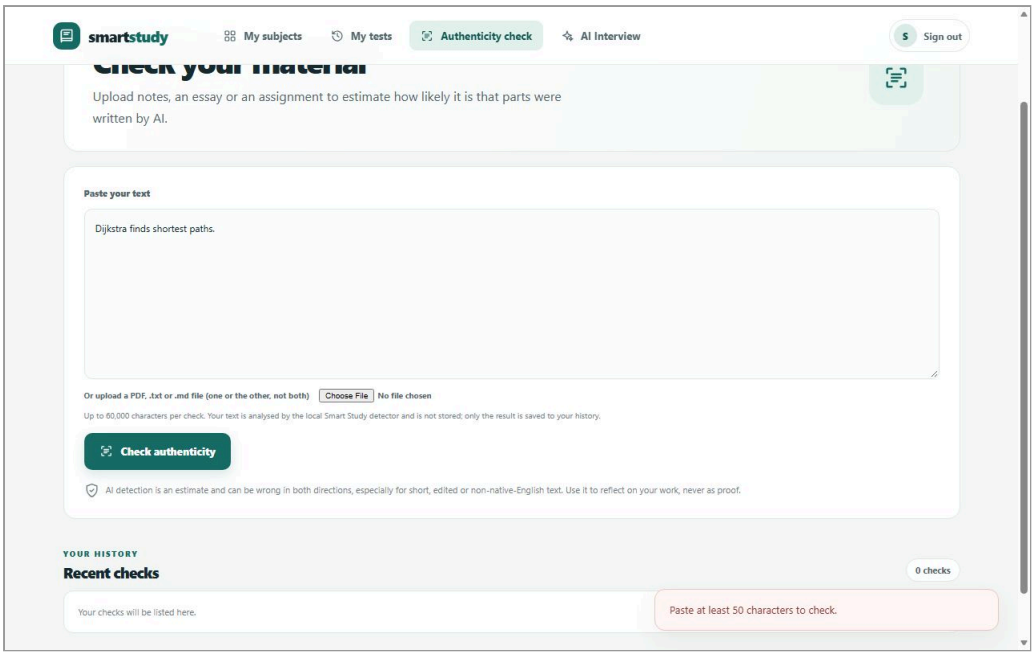


Figure 4.37: Text shorter than 50 characters: the error message appears at the bottom right.

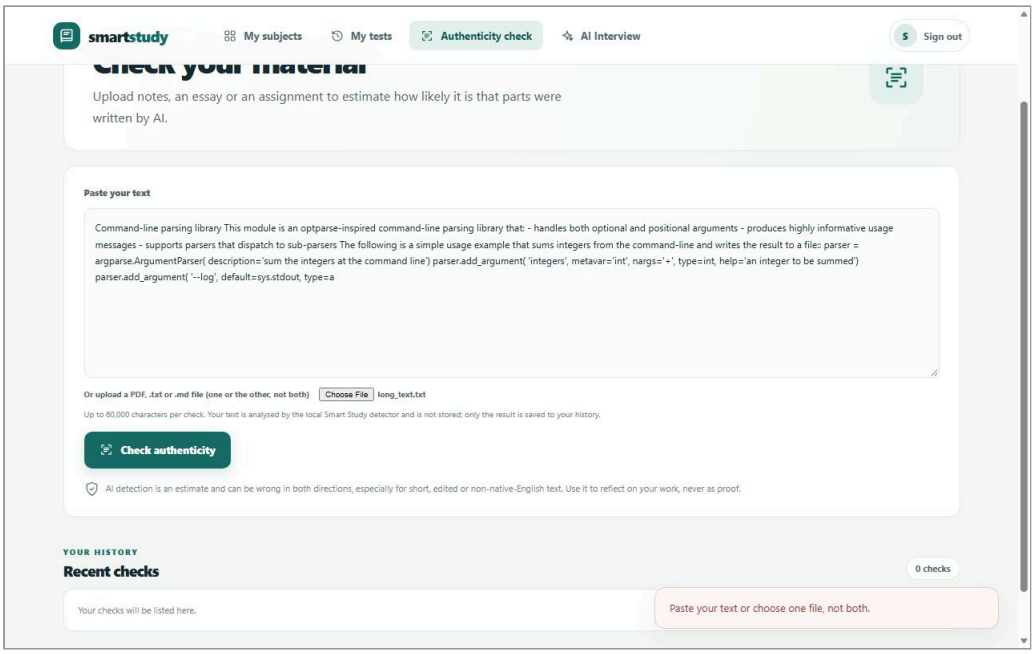


Figure 4.38: Pasted text and a file together: the error message appears at the bottom right.

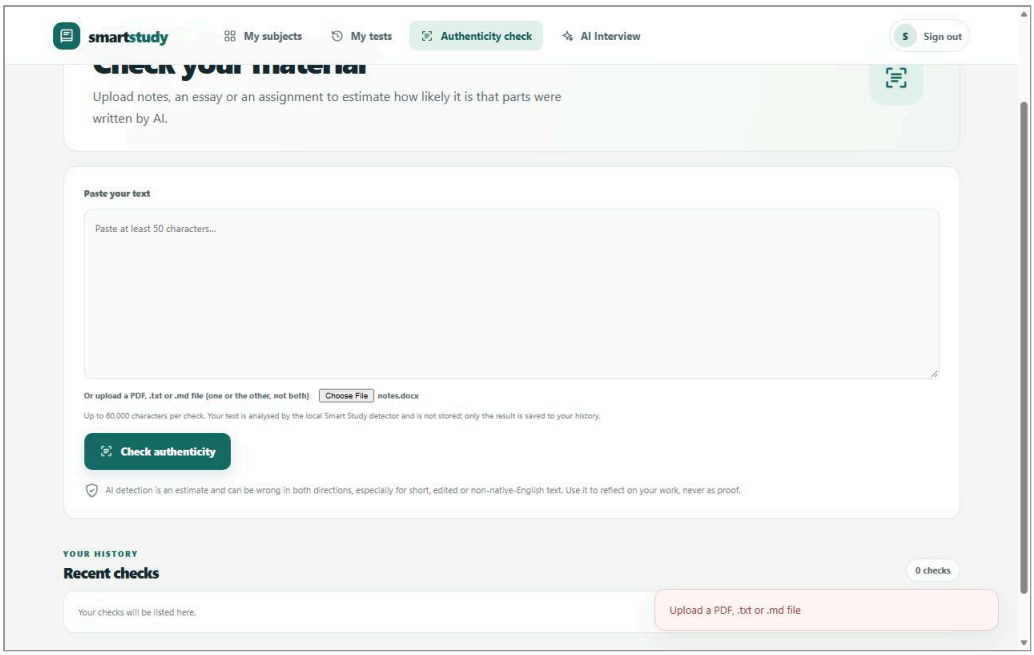


Figure 4.39: An unsupported file type: the server's error message appears at the bottom right.

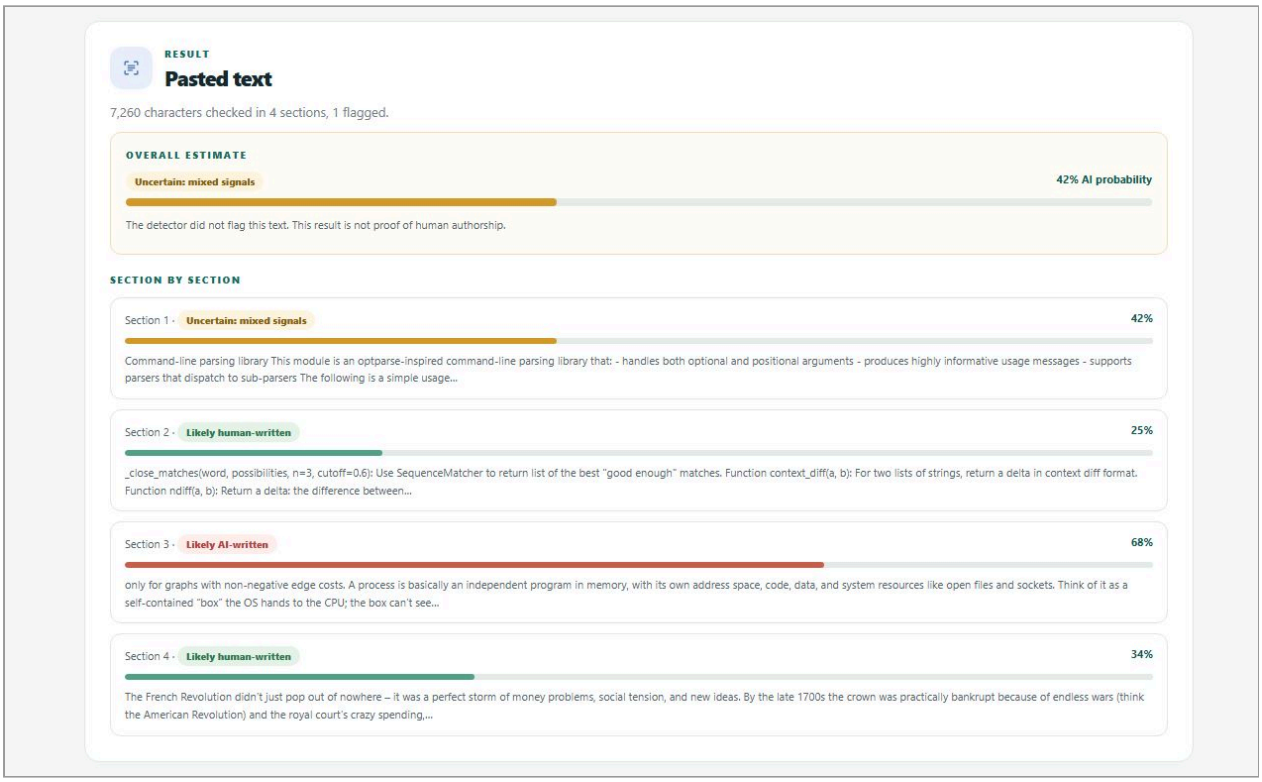


Figure 4.40: Result for a 7,260-character document: overall estimate and section-by-section scores.

The authenticity check accepts pasted text or one PDF, .txt or .md file. Long text is split into sections that are scored separately, and the overall estimate is shown with a label: likely human-written (below 35%), uncertain (35–60%) or likely AI-written (60% and above). The test document mixed human-written documentation passages with AI-generated answers.

Test	Test case	Input	System output	Result
Test 29	Nothing entered	Text: (empty), no file	Error message “Paste your text or choose one file, not both.” (Figure 4.36).	Pass
Test 30	Text shorter than 50 characters	Text: “Dijkstra finds shortest paths.”	Error message “Paste at least 50 characters to check.” (Figure 4.37).	Pass
Test 31	Both text and a file	Text: 600 characters File: long_text.txt	Error message “Paste your text or choose one file, not both.” (Figure 4.38).	Pass
Test 32	Unsupported file type	File: notes.docx	Error message “Upload a PDF, .txt or .md file” (Figure 4.39).	Pass
Test 33	Valid document	Text: 7,260 characters (human and AI passages mixed)	“Uncertain: mixed signals”, 42% overall, 4 sections with 1 flagged as likely AI-written (68%); added to Recent checks (Figure 4.40).	Pass

AI Interview Practice

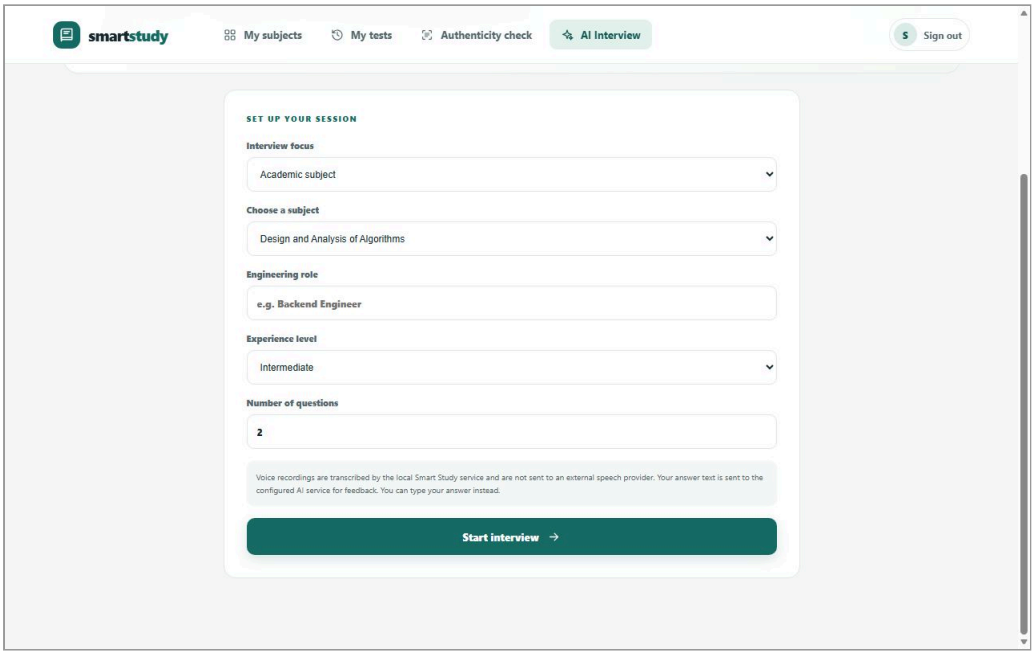


Figure 4.41: Interview set-up: academic subject, experience level and number of questions.

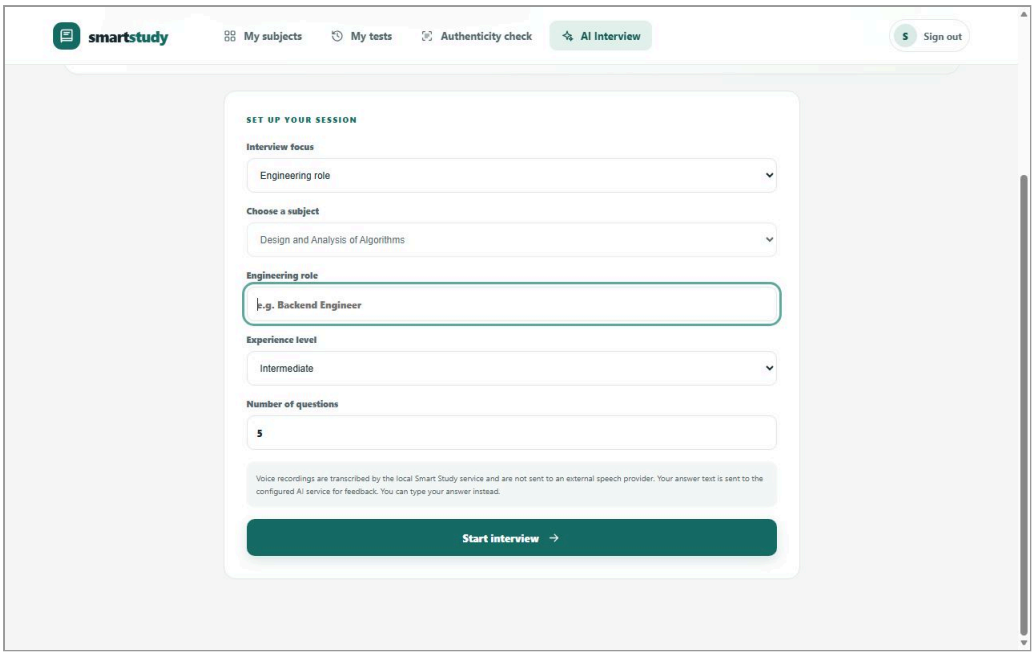


Figure 4.42: Engineering role left empty: the browser blocks the form and shows “Please fill out this field.” next to the field.

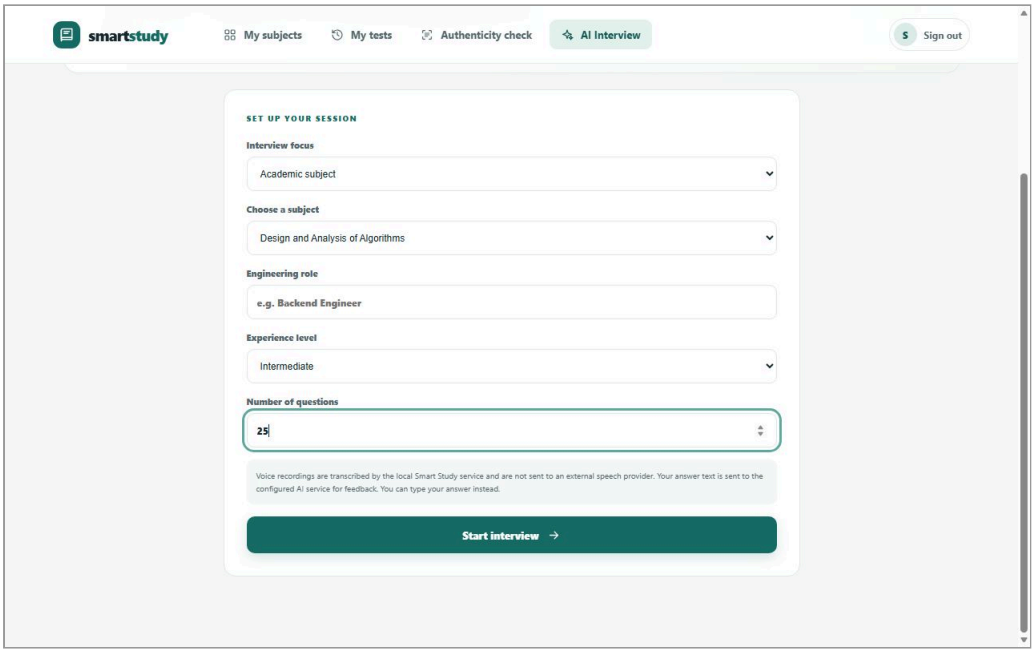


Figure 4.43: 25 questions requested: the browser blocks the form and shows “Value must be less than or equal to 20.” next to the field.

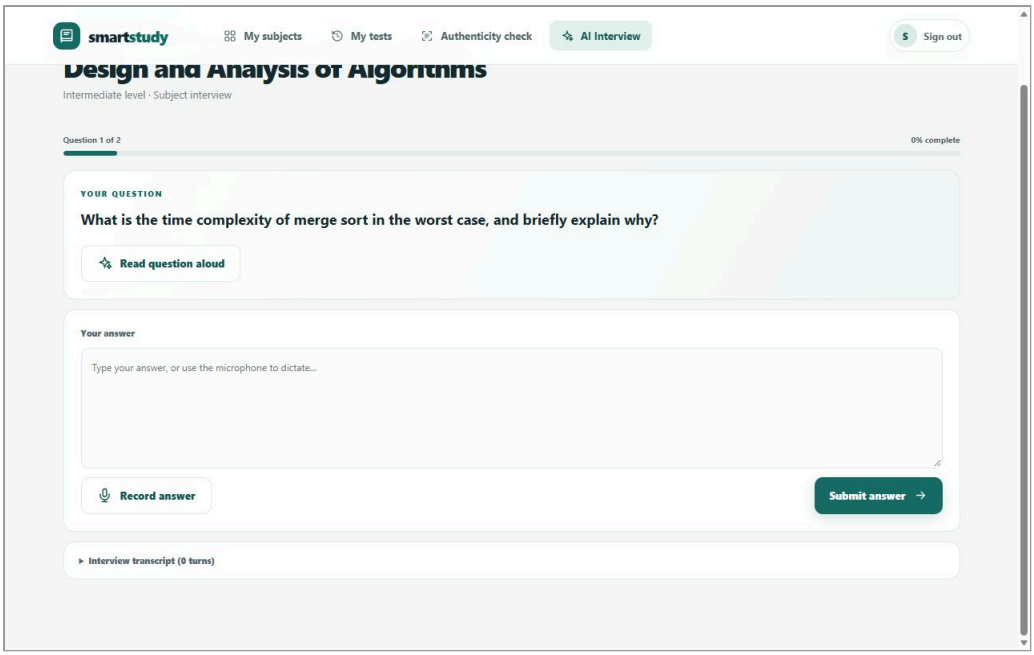


Figure 4.44: The interviewer's first question.

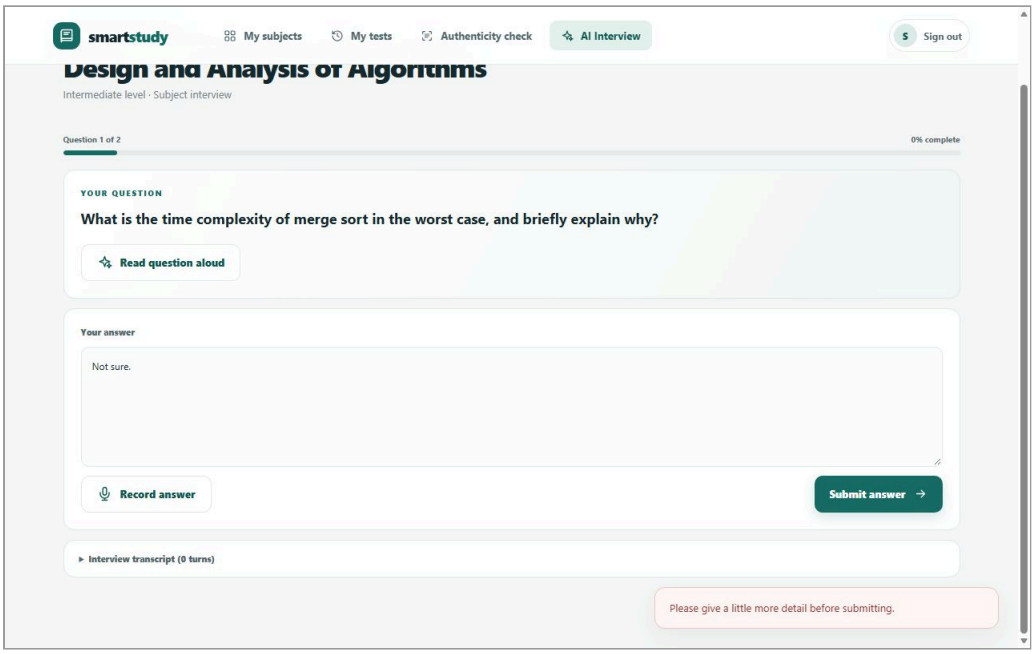


Figure 4.45: An answer with too little detail: the error message appears at the bottom right.

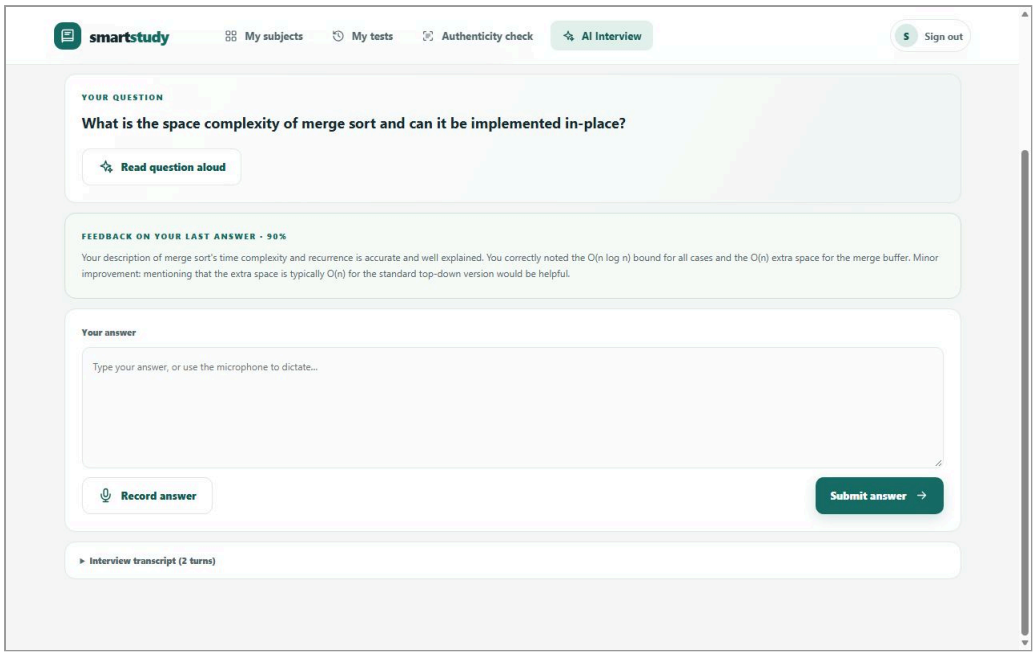


Figure 4.46: Feedback on the previous answer and the next question.

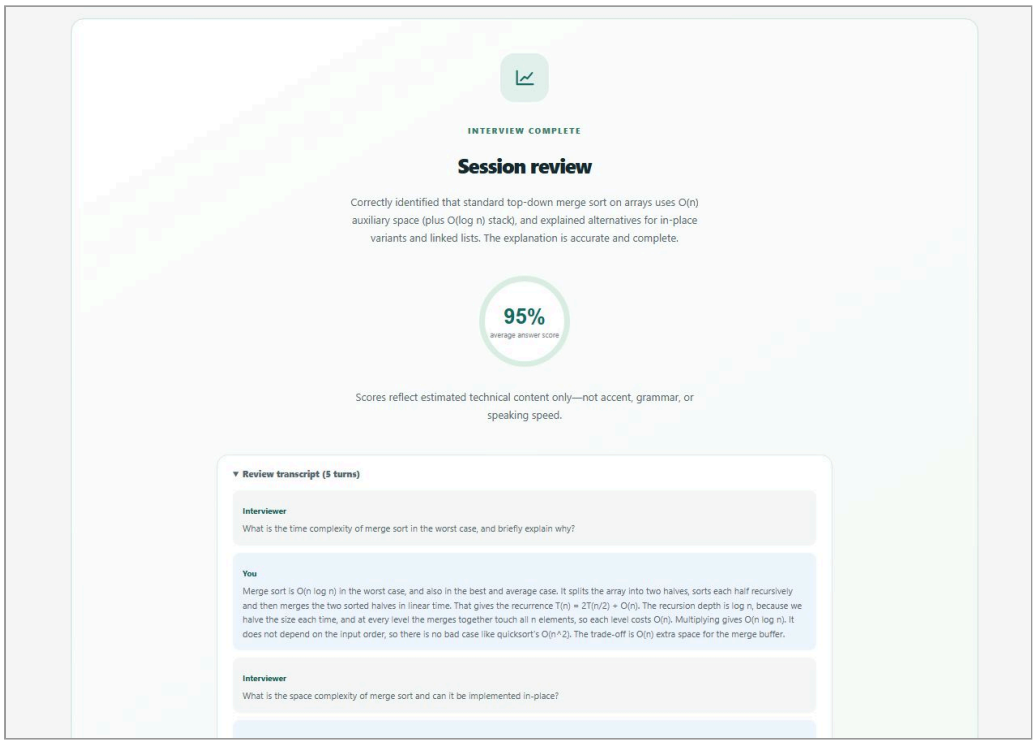


Figure 4.47: Session review at the end of the interview, with the transcript.

The learner chooses an academic subject or an engineering role, the experience level and the number of questions, then answers each question by typing or by recording their voice. After each answer the interviewer gives feedback and asks the next question; at the end a session review shows the average score and the full transcript.

Test	Test case	Input	System output	Result
Test 34	Engineering role left empty	Focus: Engineering role Role: (empty)	Browser blocks the form and shows "Please fill out this field."; nothing is sent to the server (Figure 4.42).	Pass
Test 35	Too many questions	Number of questions: 25	Browser blocks the form and shows "Value must be less than or equal to 20."; nothing is sent to the server (Figure 4.43).	Pass
Test 36	Start an interview	Focus: Academic subject (Design and Analysis of Algorithms) Level: Intermediate Questions: 2	First question shown: "What is the time complexity of merge sort in the worst case, and briefly explain why?" (Figures 4.41 and 4.44).	Pass
Test 37	Answer with too little detail	Answer: "Not sure."	Error message "Please give a little more detail before submitting." (Figure 4.45).	Pass
Test 38	Answer the first question	Answer explaining the O(n log n) bound and the recurrence T(n) = 2T(n/2) + O(n)	Feedback scored 90% and the next question on merge sort's space complexity (Figure 4.46).	Pass
Test 39	Answer the last question	Answer on O(n) auxiliary space and in-place variants	Session review with 95% average answer score and the transcript (Figure 4.47).	Pass

Test Cases Verified by Automated Tests

Some behaviour cannot be shown on a screen, or depends on a service failing. These cases are covered by the automated suites listed above.

Test	Test case	Input	System output	Result
Test 40	Another user's data through the API	Request for a subject or saved test owned by another account	HTTP 403; the resource is not returned (backend integration tests).	Pass
Test 41	AI-writing detector unavailable	Written answer submitted while the detector cannot be reached	Answer is graded and saved without an AI estimate (backend quiz service unit test).	Pass
Test 42	Incorrect option is stored	Multiple-choice answer with a wrong option	The saved answer records the option the learner chose (backend quiz service unit test).	Pass
Test 43	Scanned or invalid PDF	Non-PDF data, a document over the page limit, scanned pages	Clear error, or text read by OCR in batches (ML service tests).	Pass

Test	Test case	Input	System output	Result
Test 44	Invalid model output	Language-model response with missing fields or a wrong number of options	Invalid items are rejected and never shown as questions (ML service tests).	Pass
Test 45	ML service drops the connection	Connection reset while the backend waits for a grade	Request retried once on a new connection and the grade returned; a second failure gives “The ML service closed the connection. Please try again.” (backend ML client tests).	Pass
Test 46	Invalid input sent straight to the API	Password “abc”, email “notanemail”, blank subject or topic name	HTTP 400 with “Password must be at least 6 characters”, “Enter a valid email address” or “Enter a subject name (up to 255 characters)” (backend integration tests).	Pass

AI-writing detector evaluation

To check whether the AI-writing estimate is meaningful for current language models, eight answers were generated with gpt-oss:120b (four in an exam style and four in a casual student voice, on topics such as Dijkstra’s algorithm, TCP and photosynthesis) and compared with eight human-written passages taken from Python standard-library documentation. Each detector’s AI probability was compared with the 0.60 flag threshold.

Detector	AI texts flagged (of 8)	Human texts wrongly flagged (of 8)	Model size
Hello-SimpleAI/chatgpt-detector-roberta (2023, previously used)	0	0	about 500 MB
fakespot-ai/roberta-base-ai-text-detection-v1 (2025, adopted)	3	2	499 MB
MayZhou/e5-small-lora-ai-generated-detector	2	4	133 MB

The original detector, trained on 2023 ChatGPT answers [16], did not flag any of the eight current-model answers: every AI sample scored below 4%. The Fakespot model was adopted because it detected the largest share of AI text while wrongly flagging fewer human passages than the e5-small model. On a 7,260-character document mixing three human passages and four AI answers, it rated one AI-written section at 68% and flagged none of the human sections, but it rated another AI-written section at only 34%. Its output can therefore be wrong in both directions, which is why the application presents it as an advisory estimate. Sixteen samples are too few for reliable accuracy figures, and documentation prose differs from student writing; a larger evaluation is listed as future work.

CHAPTER 5
CONCLUSION

Conclusion

Smart Study demonstrates an integrated web application for organizing examination preparation around learner-supplied subject material. Its implemented workflow connects account-scoped subjects and topics, PDF paper ingestion, question and recurrence analysis, generated practice, adaptive tests by question type, stored test history for self-evaluation, subjective feedback with an advisory AI-writing estimate, a standalone authenticity check and interview rehearsal. A Java API coordinates persistent domain operations, while a Python ML service isolates text analysis, language-model interaction, OCR and speech transcription. Docker Compose describes a repeatable local multi-service deployment.

The project also demonstrates how a simple ability estimate can influence practice selection. The implemented logistic update and nearest-difficulty selection make quiz sessions responsive to learner answers, while remaining intentionally simpler than a fully calibrated item-response assessment. Fresh automated runs on 27 October 2026 passed all 67 ML-service tests and all 53 backend tests, and end-to-end runs against the deployed services confirmed question generation, adaptive practice, written-answer grading, test review, the authenticity check and voice transcription. Live testing also exposed real problems that were then fixed: the hosted model ignored the requested JSON format, first-time model downloads could block requests, the original AI-writing detector flagged none of the current-model answers, loading two models at the same time left the detector unusable, a dropped connection to the ML service could fail the grading of a written answer, the API accepted invalid registration and subject input that only the browser had checked, and generated explanations referred to options by positions that changed when the options were shuffled. These results support important implementation paths; they do not establish educational effectiveness, model accuracy, accessibility compliance, load capacity or production readiness.

The main practical value is workflow integration rather than a claim that automated generation or clustering is inherently superior to instructor-authored learning. Uploaded paper quality, correct user configuration, reliable model responses, review of generated items and secure operations remain necessary. Wider use would require verifying the deployment with an approved model-provider configuration, adding explicit authorization and failure-path tests, and reviewing the quality of generated questions with educators.

Limitations

- Clustering combines TF-IDF with MiniLM embeddings, which groups reworded questions better but can still merge different questions on a shared topic. Recurring-question detection

uses TF-IDF only and may miss paraphrases. Cluster labels are descriptive terms, not validated syllabus topics.

- AI-writing detection is unreliable: in a 16-sample evaluation the adopted detector flagged 3 of 8 AI-written answers and wrongly flagged 2 of 8 human passages. Its estimate must never be treated as proof, and the sample is too small to give a dependable accuracy figure.
- The hosted language model did not enforce the requested JSON schema; the service compensates with prompting, lenient parsing and validation, but some generated items may still be discarded and a request can fail.
- Question generation and subjective grading depend on an external language model and can be incorrect, inconsistent or biased.
- The adaptive algorithm is a simplified pedagogical heuristic; it has not been calibrated on a representative learner/item dataset.
- OCR accuracy varies with scan quality, layout and language; extraction should be inspected before relying on results.
- No controlled learner study, formal accessibility review, production load test or availability evaluation is included in the current evidence.
- The current local deployment requires operator-provided secrets, database setup and model-service configuration. The local models total about 735 MB and can take a long time to download on slow connections.

Future Scope

- Conduct educator and learner evaluation; measure task completion, usefulness, question quality and learning outcomes.
- Add instructor review/approval workflows and citations to the uploaded source passages behind generated questions.
- Calibrate embedding thresholds on a larger labelled set before extending embeddings to recurring-question detection, while retaining transparent thresholds and human review.
- Evaluate AI-writing detection on a larger, consented set of student answers; compare stronger detectors and report uncertainty for each section.
- Use the saved test history to show progress over time for each topic.
- Evaluate calibrated IRT models using sufficient response data, item calibration, uncertainty estimates and privacy-preserving analytics.
- Add versioned database migrations, retention controls, backups, accessibility testing, security assessment and operational monitoring.
- Support more document types and languages after measuring extraction accuracy and defining data-handling requirements.

REFERENCES

1. F. M. Lord, M. R. Novick and A. Birnbaum, *Statistical Theories of Mental Test Scores*, Addison-Wesley, 1968.
2. F. B. Baker and S.-H. Kim, *Item Response Theory: Parameter Estimation Techniques*, 2nd ed., Marcel Dekker, 2004.
3. G. Salton and M. J. McGill, *Introduction to Modern Information Retrieval*, McGraw-Hill, 1983.
4. J. MacQueen, "Some Methods for Classification and Analysis of Multivariate Observations," in *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, vol. 1, pp. 281–297, 1967.
5. P. J. Rousseeuw, "Silhouettes: A Graphical Aid to the Interpretation and Validation of Cluster Analysis," *Journal of Computational and Applied Mathematics*, vol. 20, pp. 53–65, 1987.
6. Moodle Documentation, "Quiz activity." Available at: https://docs.moodle.org/en/Quiz_activity. Accessed 25 October 2026.
7. Quizlet, "Study with flashcards and learning tools." Available at: https://quizlet.com/. Accessed 25 October 2026.
8. Khan Academy, "Practice." Available at: https://www.khanacademy.org/. Accessed 25 October 2026.
9. Spring, "Spring Boot Reference Documentation." Available at: https://docs.spring.io/spring-boot/reference/. Accessed 25 October 2026.
10. FastAPI, "FastAPI Documentation." Available at: https://fastapi.tiangolo.com/. Accessed 25 October 2026.
11. PostgreSQL Global Development Group, "PostgreSQL Documentation." Available at: https://www.postgresql.org/docs/. Accessed 25 October 2026.
12. scikit-learn developers, "Clustering" and "Feature extraction." Available at: https://scikit-learn.org/stable/. Accessed 25 October 2026.
13. Ollama, "Ollama API documentation." Available at: https://docs.ollama.com/. Accessed 25 October 2026.
14. SYSTRAN, "Faster-Whisper: Faster Whisper transcription with CTranslate2." Available at: https://github.com/SYSTRAN/faster-whisper. Accessed 25 October 2026.
15. Project source repository, *Smart Study*: README, backend source, frontend source, ML service source, and automated tests. Repository workspace reviewed on 26 October 2026.
16. B. Guo, X. Zhang, Z. Wang, M. Jiang, J. Nie, Y. Ding, J. Yue and Y. Wu, "How Close is ChatGPT to Human Experts? Comparison Corpus, Evaluation, and Detection,"

arXiv:2301.07597, 2023.

17. W. Liang, M. Yuksekgonul, Y. Mao, E. Wu and J. Zou, “GPT detectors are biased against non-native English writers,” *Patterns*, vol. 4, no. 7, 100779, 2023.
18. Fakespot (Mozilla), “roberta-base-ai-text-detection-v1” model card. Available at: https://huggingface.co/fakespot-ai/roberta-base-ai-text-detection-v1. Accessed 26 October 2026.
19. N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” in *Proceedings of EMNLP-IJCNLP 2019*, pp. 3982–3992, 2019.
20. W. Wang, F. Wei, L. Dong, H. Bao, N. Yang and M. Zhou, “MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers,” in *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, 2020.

APPENDIX

A. Abbreviation List

Abbreviation	Full form
AI	Artificial Intelligence
API	Application Programming Interface
CPU	Central Processing Unit
CSS	Cascading Style Sheets
DTO	Data Transfer Object
E-R / ERD	Entity-Relationship (Diagram)
FK	Foreign Key
GB / MB	Gigabyte / Megabyte
GPU	Graphics Processing Unit
HOD	Head of Department
HTML	HyperText Markup Language
HTTP / HTTPS	HyperText Transfer Protocol (Secure)
IRT	Item Response Theory
JDK	Java Development Kit
JPA	Jakarta Persistence API
JSON	JavaScript Object Notation
JWT	JSON Web Token
LLM	Large Language Model
ML	Machine Learning
MVC	Model–View–Controller
OCR	Optical Character Recognition
PDF	Portable Document Format
PK	Primary Key
RAM	Random Access Memory
REST	Representational State Transfer
RUP	Rational Unified Process

Abbreviation	Full form
SQL	Structured Query Language
SSD	Solid State Drive
TC	Test Case
TCP	Transmission Control Protocol
TF-IDF	Term Frequency–Inverse Document Frequency
UC	Use Case
UI	User Interface
UML	Unified Modeling Language
URL	Uniform Resource Locator

B. User Manual

B.1 Installing and starting the application

1. Install Docker Desktop (Linux containers) and start it. Obtain an API key for the configured Ollama model endpoint.
2. Copy `.env.example` to `.env` and `ml_services\.env.example` to `ml_services\.env`. Set new values for `POSTGRES_PASSWORD`, `JWT_SECRET` and `ML_INTERNAL_KEY` in `.env`, and set `OLLAMA_API_KEY` in `ml_services\.env`.
3. From the `Frontend` folder run `docker compose --env-file ..\.env up --build -d`. The first build takes 15–30 minutes; the ML service then downloads its local models (about 750 MB) in the background.
4. Open `http://localhost:3000` (or the port set in `FRONTEND_PORT`) in a browser.
5. To stop the application while keeping all data, run `docker compose --env-file ..\.env down`.

B.2 Using Smart Study

1. **Create an account and sign in.** Choose *Create account*, enter your name, email and password, then use *Sign in* on later visits.
2. **Add a subject.** On the dashboard choose *Add subject* and enter its name. Open the subject to work with it.
3. **Upload past papers and build a practice set.** On the subject page, under *Build a practice set*, click the upload area and choose one to five PDF question papers. Enter the number of questions (1–50) and the question format (multiple choice, subjective or both), then choose

Generate my questions. The uploaded papers, the topics identified in them and the generated questions are shown when processing finishes.

4. **Take a test.** Under *Take a test from your question bank*, choose the question type and number of questions and choose *Start test*. Select an option or type a written answer and choose *Submit answer*; feedback is shown before *Next question*. A written answer can also be uploaded as a PDF.
5. **Review past tests.** Open *My tests* and select a test to see each question, your answer, the correct option or model answer, the grade, feedback and AI-writing estimate.
6. **Check material authenticity.** Open *Authenticity check*, paste text or upload a PDF, .txt or .md file (50–60,000 characters) and choose *Check authenticity*. The overall estimate, the section-by-section scores and your recent checks are shown.
7. **Practice an interview.** Open *AI Interview*, choose an academic subject or an engineering role, the level and the number of questions, and choose *Start interview*. Type an answer, or choose *Record answer* and allow microphone access to dictate it, then submit it to receive feedback and the next question. Choose *End interview* to finish early.
8. **Sign out** from the navigation bar when finished.

B.3 Common problems

- *The AI-writing check or voice dictation says the model is still loading:* the first-start model download is still running; wait and try again.
- *Question generation fails:* check that `OLLAMA_API_KEY` is set and the model endpoint is reachable.
- *No text is found in a paper:* the PDF may be a poor-quality scan; try a clearer copy.
- *The microphone cannot be used:* open the application on `localhost` or over HTTPS and allow microphone access, or type the answer instead.

C. Glossary

Term	Meaning
Ability estimate	A number that represents how well the learner is currently answering. It rises after correct answers and falls after incorrect ones, and the next question is chosen with a difficulty close to it.
Adaptive quiz	A test in which the next question depends on the learner's previous answers instead of following a fixed order.
AI-writing check	An estimate, produced by a local classifier, of the probability that a piece of text was written by an AI model. It is advisory and never changes a grade.
Authenticity check	The Smart Study page where a learner pastes text or uploads a file to receive an AI-writing estimate with a section-by-section breakdown.

Term	Meaning
Burstiness	How much sentence length varies across a text. Human writing usually varies more than AI-generated text, so it is used as a supporting signal.
Clustering	Grouping past-paper questions that are similar in wording or meaning so that recurring topics can be identified.
Cosine similarity	A measure of how similar two text vectors are, from 0 (unrelated) to 1 (identical direction).
Docker Compose	A tool that starts all the services of the application (frontend, backend, ML service and database) together from one configuration file.
Docker volume	Storage managed by Docker that keeps data such as the database, uploaded papers and downloaded models when containers are rebuilt or restarted.
Embedding	A list of numbers that represents the meaning of a sentence, so that sentences with similar meaning have similar embeddings.
Join table	A table that links two other tables in a many-to-many relationship, such as `quiz_session_questions`.
Marking guide	The points a good written answer should cover, used together with the model answer to grade a subjective answer.
Question bank	All the questions saved for a subject, from which tests are drawn.
Subjective question	A question answered in the learner's own words rather than by choosing an option.
Token	A small unit of text (a word or part of a word) that a language model reads. The AI-writing detector scores text in windows of 512 tokens.
Transcription	Converting recorded speech into text. Smart Study uses a local Whisper model for this.

D. API Summary

Route family	Purpose
`/api/auth/register`, `/api/auth/login`	Account registration and login.
`/api/subjects`, `/api/subjects/{id}/topics`	Subject and topic organization.
`/api/subjects/{id}/papers`, `/api/papers/{id}/text`	Paper upload, listing, extracted text and deletion.
`/api/subjects/{id}/analysis`	Past-paper question analysis.
`/api/subjects/{id}/questions/generate-from-papers`	Generate questions using subject paper material.
`/api/topics/{id}/questions/generate`	Generate questions for a topic.

Route family	Purpose
`/api/subjects/{id}/quiz`, `/api/quiz/{sessionId}/answer`, `/api/quiz/{sessionId}/subjective-answer`	Start a test (optional `questionType`: `multiple_choice`, `subjective` or `mixed`) and answer it; written answers return a grade, feedback and AI-writing estimate.
`/api/subjects/{id}/question-bank`	Count of multiple-choice and subjective questions available for practice.
`/api/quizzes`, `/api/quizzes/{sessionId}`	List saved tests and review one test question by question.
`/api/authenticity/check`, `/api/authenticity/checks`	Check pasted text or an uploaded PDF/.txt/.md file for AI-written content; list recent checks.
`/api/interview/turn`, `/api/interview/transcribe`	Interview question/feedback turn and authenticated transcription relay.

Exact request schemas, authorization rules and error responses are defined by the current backend controllers, DTOs and ML service schemas; consult source/API documentation before integrating another client.